

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ
MÔN ĐỒ ÁN TỐT NGHIỆP

Đề tài: Xây dựng khung học đa phương thức trong chẩn đoán bệnh lý xương, kết hợp dữ liệu hình ảnh y khoa và các chỉ số lâm sàng

GVHD: ThS. Đỗ Minh Tiến

Sinh viên thực hiện:

- | | |
|-------------------|----------------|
| 1. Dương Phú Vinh | MSSV: 24410376 |
| 2. Vũ Tiến Lập | MSSV: 24410316 |

[illegible]

Người nhận xét
(Ký tên và ghi rõ họ tên)

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp với đề tài "**Xây dựng khung học đa phương thức trong chẩn đoán bệnh lý xương, kết hợp dữ liệu hình ảnh y khoa và các chỉ số lâm sàng**" là kết quả nghiên cứu, tìm hiểu và phát triển của bản thân em dưới sự hướng dẫn của giảng viên hướng dẫn. Các nội dung trình bày trong báo cáo được tổng hợp từ quá trình phân tích yêu cầu, thiết kế, cài đặt và kiểm thử hệ thống. Những tài liệu, công nghệ, thư viện và phương pháp tham khảo đều được trình bày trong phần tài liệu tham khảo.

Em xin chịu trách nhiệm về tính trung thực của nội dung báo cáo này. Hệ thống trong đồ án chỉ phục vụ mục đích học tập, nghiên cứu và minh họa kỹ thuật, không được sử dụng để thay thế chẩn đoán hoặc chỉ định điều trị của bác sĩ.

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô trong khoa đã trang bị cho em kiến thức nền tảng về lập trình, cơ sở dữ liệu, trí tuệ nhân tạo, phát triển phần mềm và phương pháp nghiên cứu trong suốt quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn đã định hướng đề tài, góp ý về phạm vi hệ thống, kiến trúc, cách đánh giá mô hình và các yêu cầu cần chú ý khi áp dụng AI trong lĩnh vực y tế.

Em cũng xin cảm ơn gia đình, bạn bè và các anh chị đã hỗ trợ, động viên trong quá trình thực hiện đồ án. Trong quá trình xây dựng hệ thống, do giới hạn về thời gian, dữ liệu y khoa có nhãn và điều kiện triển khai thực tế, đồ án chắc chắn còn nhiều thiếu sót. Em rất mong nhận được sự góp ý của quý thầy cô để hệ thống được hoàn thiện hơn trong tương lai.

TÓM TẮT

Áp dụng Trí tuệ Nhân tạo (AI) vào y tế đối mặt với một thách thức lớn: làm sao để khai thác sức mạnh của mô hình ngôn ngữ và thị giác máy tính mà vẫn kiểm soát được tính an toàn, tránh tình trạng "ảo giác" thông tin. Đồ án này trình bày quá trình nghiên cứu và xây dựng một hệ thống trợ lý y khoa đa phương thức dạng web, nhằm giải quyết bài toán truy xuất thông tin và hỗ trợ sàng lọc bệnh lý. Hệ thống được thiết kế theo kiến trúc phân lớp chặt chẽ, kết hợp Next.js và FastAPI, sử dụng Model Context Protocol (MCP) làm cầu nối giao tiếp.

Thay vì chỉ là một chatbot thông thường, cốt lõi của hệ thống nằm ở cơ chế RAG (Retrieval-Augmented Generation) đã được tinh chỉnh qua các bộ lọc chủ đề nhằm chặn đứng lỗi sai lệch ngữ nghĩa. Đồng thời, đồ án xây dựng một pipeline hoàn chỉnh để xử lý và phân tích ảnh y khoa (X-quang/CT), kết hợp với module đọc chỉ số xét nghiệm và thuật toán fusion để đánh giá chéo các nguồn dữ liệu lâm sàng. Việc tích hợp cơ chế hiệu chỉnh độ tin cậy (confidence calibration) và thiết lập ranh giới "Analysis Only" đảm bảo AI chỉ đóng vai trò hỗ trợ, kiên quyết từ chối đưa ra chẩn đoán khi dữ liệu không đủ an toàn. Kết quả thực nghiệm thông qua các chỉ số như Macro-F1 đã chứng minh hệ

thông hoạt động ổn định, mở ra hướng triển khai khả thi khi có bộ dữ liệu lâm sàng quy mô lớn.

MỤC LỤC

LỜI CAM ĐOAN.....	1
LỜI CẢM ƠN.....	2
TÓM TẮT.....	2
MỤC LỤC	4
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	9
1.1. Lý do chọn đề tài	9
1.2. Mục tiêu đề tài	10
1.3. Đối tượng và phạm vi nghiên cứu	11
1.4. Phương pháp thực hiện.....	12
1.5. Ý nghĩa thực tiễn của đề tài.....	12
1.6. Cấu trúc báo cáo	12
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	14
2.1. Trí tuệ nhân tạo trong hỗ trợ y khoa.....	14
2.2. Mô hình ngôn ngữ lớn và chatbot	14
2.3. Retrieval-Augmented Generation.....	14
2.4. Model Context Protocol	15
2.5. Xử lý ảnh y tế	15
2.6. Phân tích phim xương bằng mô hình AI	16
2.7. Phân tích xét nghiệm	16
2.8. Fusion đa phương thức	17
2.9. Đánh giá mô hình	17
2.10. Công nghệ sử dụng.....	17
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU HỆ THỐNG	19
3.1. Bài toán đặt ra.....	19

3.2. Tác nhân sử dụng hệ thống.....	19
3.3. Yêu cầu chức năng	19
3.3.1. Chức năng chatbot.....	19
3.3.2. Chức năng phân tích phim xương	20
3.3.3. Chức năng xét nghiệm.....	20
3.3.4. Chức năng fusion.....	21
3.3.5. Chức năng đánh giá mô hình.....	21
3.4. Yêu cầu phi chức năng	22
3.4.1. Tính dễ sử dụng	22
3.4.2. Tính phản hồi.....	22
3.4.3. Tính mở rộng	22
3.4.4. Tính an toàn.....	23
3.4.5. Tính kiểm thử	23
3.5. Use case tổng quát	23
3.5.1. Use case hỏi bệnh lý	23
3.5.2. Use case phân tích phim xương.....	23
3.5.3. Use case phân tích xét nghiệm	24
3.5.4. Use case fusion	24
3.5.5. Use case đánh giá	24
CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU	25
4.1. Kiến trúc tổng thể	25
4.2. Thiết kế frontend	25
4.3. Thiết kế backend y khoa.....	26
4.4. Thiết kế cơ sở dữ liệu Chat.....	26
4.5. Thiết kế dữ liệu backend y khoa.....	27
4.6. Thiết kế API.....	28

4.7. Thiết kế luồng chatbot RAG/MCP	29
4.8. Thiết kế luồng phân tích ảnh	30
4.9. Thiết kế quy tắc an toàn cho model ảnh	30
CHƯƠNG 5. XÂY DỰNG CÁC MODULE CHỨC NĂNG	32
5.1. Module giao diện chatbot	32
5.2. Module nhận diện câu hỏi y khoa.....	33
5.3. Module truy xuất dữ liệu nội bộ	33
5.4. Module upload phim xương	34
5.5. Pipeline xử lý ảnh	35
5.6. Module model ảnh	36
5.7. Module xét nghiệm.....	36
5.8. Module lâm sàng	38
5.9. Module fusion đa phương thức.....	38
5.10. Module đánh giá mô hình.....	39
5.11. Module xác thực và lịch sử.....	40
CHƯƠNG 6. HUẤN LUYỆN, SUY LUẬN VÀ ĐÁNH GIÁ MÔ HÌNH AI	42
6.1. Vai trò của model AI train trong hệ thống	42
6.2. Dữ liệu huấn luyện demo	42
6.3. Huấn luyện lightweight feature model	43
6.4. Huấn luyện CNN/TorchScript model.....	43
6.5. Hiệu chỉnh confidence	44
6.6. Quy tắc model_prediction và analysis_only	44
6.7. Đánh giá model bằng metrics	45
6.8. Đánh giá chatbot RAG	45
6.9. Đánh giá fusion.....	46
6.10. Giới hạn của model trong đồ án	46

CHƯƠNG 7. THỬ NGHIỆM HỆ THỐNG VÀ KẾT QUẢ ĐẠT ĐƯỢC	47
7.1. Môi trường thử nghiệm	47
7.2. Kiểm thử chatbot	47
7.3. Kiểm thử phân tích phim xương.....	48
7.4. Kiểm thử xét nghiệm	48
7.5. Kiểm thử fusion	49
7.6. Kiểm thử đánh giá model	49
7.7. Kiểm thử kỹ thuật	50
7.8. Kết quả đạt được.....	50
7.9. Hạn chế	50
CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	52
8.1. Kết luận.....	52
8.2. Đóng góp của đề án	52
8.3. Hướng phát triển.....	53
8.4. Kết luận cuối.....	53
TÀI LIỆU THAM KHẢO	54
PHỤ LỤC A. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY HỆ THỐNG	55
A.1. Chạy PostgreSQL	55
A.2. Chạy Chat frontend.....	55
A.3. Chạy MCP server.....	55
A.4. Chạy FastAPI backend trong Git Bash.....	55
A.5. Cấu hình biến môi trường Chat	55
PHỤ LỤC B. MẪU CSV ĐÁNH GIÁ.....	57
PHỤ LỤC C. GỢI Ý DANH MỤC HÌNH ẢNH CẦN CHÈN.....	58
PHỤ LỤC D. GỢI Ý BẢNG BIỂU.....	59
PHỤ LỤC E. GHI CHÚ AN TOÀN Y KHOA.....	60

PHỤ LỤC F. ĐẶC TẢ USE CASE CHI TIẾT	61
F.1. Use case UC01 - Đăng ký tài khoản y khoa.....	61
F.2. Use case UC02 - Đăng nhập hệ thống y khoa.....	61
F.3. Use case UC03 - Hỏi thông tin bệnh lý bằng chatbot.....	61
F.4. Use case UC04 - Upload và phân tích phim xương	62
F.5. Use case UC05 - Xem lịch sử upload phim xương.....	62
F.6. Use case UC06 - Nhập và đọc kết quả xét nghiệm	63
F.7. Use case UC07 - Upload file xét nghiệm.....	63
F.8. Use case UC08 - Chạy fusion đa phương thức	63
F.9. Use case UC09 - Đánh giá model bằng CSV	64
F.10. Use case UC10 - Lưu prediction run thử công.....	64
PHỤ LỤC G. KỊCH BẢN KIỂM THỬ CHI TIẾT	65
G.1. Nhóm kiểm thử chatbot	65
G.2. Nhóm kiểm thử upload ảnh	65
G.3. Nhóm kiểm thử xét nghiệm	66
G.4. Nhóm kiểm thử fusion	66
G.5. Nhóm kiểm thử đánh giá	67
G.6. Nhóm kiểm thử bảo mật và phiên đăng nhập.....	67
G.7. Nhóm kiểm thử hiệu năng cơ bản.....	67
PHỤ LỤC H. ĐẶC TẢ API TÓM TẮT	68
H.1. API kiểm tra trạng thái	68
H.2. API đăng ký	68
H.3. API đăng nhập	68
H.4. API phân tích ảnh	68
H.5. API phân tích xét nghiệm	69
H.6. API phân tích file xét nghiệm.....	69

H.7. API phân tích lâm sàng.....	69
H.8. API fusion.....	69
H.9. API đánh giá	70
H.10. API upload history.....	70
PHỤ LỤC I. QUẢN LÝ RỦI RO VÀ ĐỊNH HƯỚNG KIỂM ĐỊNH.....	71
I.1. Rủi ro chatbot trả lời sai.....	71
I.2. Rủi ro model ảnh quá tự tin.....	71
I.3. Rủi ro model ảnh quá thận trọng.....	71
I.4. Rủi ro dữ liệu huấn luyện không đại diện	72
I.5. Rủi ro bảo mật dữ liệu y tế.....	72
I.6. Định hướng kiểm định lâm sàng	72
I.7. Định hướng nâng cấp RAG.....	72
I.8. Định hướng nâng cấp fusion	73
I.9. Định hướng hoàn thiện giao diện	73
I.10. Kết luận phụ lục	73

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, trí tuệ nhân tạo, đặc biệt là AI sinh tạo, mô hình ngôn ngữ lớn và thị giác máy tính, được ứng dụng ngày càng nhiều trong các hệ thống hỏi đáp, tra cứu thông tin và hỗ trợ phân tích dữ liệu. Các mô hình này có khả năng xử lý ngôn ngữ tự nhiên tốt, hiểu được nhiều cách diễn đạt khác nhau của người dùng và tạo ra phản hồi có cấu trúc. Tuy nhiên, khi áp dụng vào lĩnh vực y tế, hệ thống AI không chỉ cần trả lời tự nhiên mà còn phải đảm bảo tính chính xác, an toàn và có giới hạn sử dụng rõ ràng.

Khác với các hệ thống hỏi đáp thông thường, chatbot y tế có mức độ rủi ro cao hơn vì thông tin sai lệch có thể ảnh hưởng trực tiếp đến nhận thức và quyết định của người dùng về sức khỏe. Một trong những vấn đề đáng chú ý của mô hình ngôn ngữ lớn là hiện tượng sinh thông tin không chính xác nhưng vẫn trình bày với giọng điệu tự tin. Trong bối cảnh y khoa, điều này có thể khiến người dùng hiểu nhầm phản hồi của hệ thống như một kết luận chuyên môn.

Xuất phát từ vấn đề trên, đề tài không đặt mục tiêu xây dựng một hệ thống AI có thể thay thế bác sĩ hoặc trả lời mọi câu hỏi y khoa. Thay vào đó, trọng tâm của đề án là thiết kế một hệ thống trợ lý y khoa có cơ chế kiểm soát ngữ cảnh, truy xuất dữ liệu nội bộ, phân tích dữ liệu đầu vào và từ chối đưa ra kết luận khi thông tin không đủ tin cậy. Cách tiếp cận này phù hợp với vai trò của một hệ thống hỗ trợ tham khảo, trong đó AI đóng vai trò hỗ trợ người dùng tiếp cận thông tin và tổng hợp dữ liệu, không thay thế chẩn đoán hoặc chỉ định điều trị của bác sĩ.

Trong phạm vi đề án tốt nghiệp, hệ thống được xây dựng theo ba hướng chính. Thứ nhất là chatbot y khoa có khả năng trả lời câu hỏi bằng tiếng Việt, kết hợp mô hình ngôn ngữ lớn với dữ liệu nội bộ thông qua pipeline RAG và Model Context Protocol. Thứ hai là module phân tích phim xương nhằm minh họa quy trình xử lý ảnh y tế, trích xuất đặc trưng và dự đoán một số nhãn tham khảo như bình thường, nghi gãy xương, viêm hoặc thoái hóa khớp, loãng xương. Thứ ba là module xét nghiệm và fusion đa phương thức nhằm kết hợp nhiều nguồn dữ liệu như ảnh, triệu chứng, tiền sử và chỉ số xét nghiệm để tạo ra đánh giá tổng hợp có cấu trúc.

Điểm quan trọng của đề tài không chỉ nằm ở việc xây dựng một hệ thống AI có thể phản hồi câu hỏi, mà còn ở việc thiết kế các quy tắc an toàn, cơ chế đánh giá mô hình, khả năng truy vết kết quả và kiến trúc mở. Nhờ đó, các thành phần demo trong hệ thống có thể được thay thế bằng mô hình hoặc dữ liệu được kiểm định tốt hơn trong tương lai.

1.2. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống prototype có thể vận hành ổn định, cho phép người dùng tương tác với các chức năng hỏi đáp bệnh lý, phân tích phim xương, đọc kết quả xét nghiệm và tổng hợp dữ liệu đa phương thức. Bên cạnh khả năng hoạt động, hệ thống cần có cơ chế kiểm soát an toàn nhằm hạn chế việc AI đưa ra phản

hồi thiếu căn cứ. Cụ thể, mình giới hạn phạm vi (scope) lại ở việc hoàn thiện lớp giao diện web để người dùng có thể tương tác mượt mà với chatbot, upload phim xương và nạp file xét nghiệm. Ở phía backend và API trung gian, hệ thống cần tích hợp được các provider mô hình ngôn ngữ thông qua API, đồng thời đóng gói pipeline RAG để truy xuất dữ liệu nội bộ phục vụ trả lời câu hỏi y khoa.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là các kỹ thuật xây dựng hệ thống AI hỗ trợ y khoa ở mức prototype, gồm chatbot, RAG, MCP, xử lý ảnh, phân tích xét nghiệm và đánh giá mô hình. Đề tài không nhằm xây dựng một hệ thống chẩn đoán y tế hoàn chỉnh để sử dụng trong bệnh viện.

Phạm vi chức năng gồm:

- Người dùng có thể chat để hỏi thông tin bệnh lý, triệu chứng, gợi ý tham khảo và lưu ý khi cần đi khám.
- Người dùng có thể upload phim xương dạng ảnh, PDF hoặc DICOM để hệ thống phân tích chất lượng, đặc trưng ảnh và đưa ra dự đoán nếu model đủ điều kiện.
- Người dùng có thể nhập thủ công hoặc upload nội dung xét nghiệm để hệ thống đọc, chuẩn hóa và phân loại chỉ số.
- Người dùng có thể chạy phân tích fusion dựa trên ảnh, triệu chứng, tiền sử và xét nghiệm.
- Người dùng có thể lưu kết quả dự đoán và đánh giá bằng file CSV hoặc dữ liệu nhập tay.

Phạm vi dữ liệu và model:

- Dữ liệu tin tức/bệnh lý nội bộ được minh họa bằng file bảng tính *tbNews_200rows.xlsx*.
- Dữ liệu hồ sơ y tế demo được minh họa bằng *tblMedicalRecord_200rows.xlsx*.
- Model ảnh trong phạm vi hiện tại là lightweight feature model và dataset demo, chưa phải model lâm sàng được kiểm định.

- Kiến trúc hỗ trợ huấn luyện CNN/TorchScript khi có dataset y khoa thật, có nhãn và quy trình kiểm định.

1.4. Phương pháp thực hiện

Thay vì áp dụng mô hình thác nước (Waterfall) cứng nhắc, quá trình thực hiện đồ án được triển khai lặp đi lặp lại theo từng module nhỏ. Giai đoạn đầu của quá trình thực hiện là phân tích yêu cầu và tách hệ thống thành các service độc lập, gồm frontend, API trung gian, MCP server, backend xử lý y khoa và lớp dữ liệu. Cách tiếp cận này giúp hệ thống dễ mở rộng và thuận tiện hơn trong quá trình kiểm thử từng module. Sau khi chốt được luồng đi của dữ liệu từ Frontend qua API trung gian xuống Backend, mình bắt tay vào dựng khung ứng dụng Chat bằng Next.js và Tailwind CSS trước để có giao diện test ngay lập tức. Sau khi giao diện cơ bản được hoàn thiện, trọng tâm được chuyển sang tích hợp pipeline RAG, cấu hình MCP server và xây dựng backend FastAPI phục vụ các chức năng xử lý ảnh, xét nghiệm và fusion. Quá trình này được thực hiện theo từng vòng thử nghiệm nhỏ để phát hiện lỗi và điều chỉnh logic xử lý.

1.5. Ý nghĩa thực tiễn của đề tài

Khi các module được tích hợp thành một hệ thống hoàn chỉnh, đề tài cho thấy tính khả thi của việc kết hợp chatbot, RAG, MCP và xử lý ảnh trong một ứng dụng hỗ trợ y khoa dạng web. Đầu tiên, nó là một bản demo sống động cho thấy việc 'lắp ghép' RAG, LLM và Computer Vision vào chung một hệ thống không phải là chuyện lý thuyết suông. Quan trọng hơn, qua quá trình debug liên tục, đồ án này vô tình phơi bày những lỗ hổng của AI mà sách vở ít khi đề cập: tình trạng chatbot bị ảo giác từ khóa, hay model ảnh quá tự tin với các dự đoán sai. Ngoài giá trị minh họa kỹ thuật, hệ thống còn có thể được xem như một khung kiến trúc ban đầu cho các ứng dụng HealthTech. Khi có dữ liệu y khoa thật và mô hình được kiểm định, các thành phần demo hiện tại có thể được thay thế hoặc nâng cấp mà không cần thay đổi toàn bộ kiến trúc hệ thống.

1.6. Cấu trúc báo cáo

Báo cáo gồm tám chương. Chương 1 trình bày tổng quan đề tài, lý do chọn đề tài, mục tiêu và phạm vi. Chương 2 trình bày cơ sở lý thuyết và công nghệ sử dụng. Chương 3 phân tích yêu cầu hệ thống. Chương 4 trình bày thiết kế kiến trúc và dữ liệu. Chương 5

mô tả quá trình xây dựng các module chức năng. Chương 6 trình bày huấn luyện, suy luận và đánh giá model. Chương 7 trình bày thử nghiệm hệ thống và kết quả đạt được. Chương 8 kết luận và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Trí tuệ nhân tạo trong hỗ trợ y khoa

Trí tuệ nhân tạo trong y tế là lĩnh vực ứng dụng các thuật toán học máy, học sâu, xử lý ngôn ngữ tự nhiên và thị giác máy tính để hỗ trợ các tác vụ như phân loại ảnh y tế, phát hiện bất thường, trích xuất thông tin từ hồ sơ bệnh án, dự đoán nguy cơ và hỗ trợ ra quyết định. Trong bối cảnh đồ án, AI được sử dụng theo vai trò hỗ trợ, không thay thế chuyên môn bác sĩ.

Một hệ thống AI y khoa cần đáp ứng các yêu cầu khác với ứng dụng thông thường. Kết quả cần có giải thích, cần chỉ rõ mức độ tin cậy, cần cảnh báo khi dữ liệu không đủ và cần tránh khẳng định quá mức. Với hình ảnh y khoa, model cần được đánh giá trên dữ liệu đại diện, có nhãn đáng tin cậy, có so sánh với chuyên gia và có phân tích sai số. Với chatbot, hệ thống cần giảm ảo giác bằng cách truy xuất nguồn dữ liệu liên quan, kiểm soát chủ đề và đưa ra khuyến cáo đi khám khi có dấu hiệu nguy hiểm.

2.2. Mô hình ngôn ngữ lớn và chatbot

Mô hình ngôn ngữ lớn là một trong những hướng phát triển nổi bật của trí tuệ nhân tạo hiện nay, đặc biệt trong các bài toán xử lý ngôn ngữ tự nhiên và xây dựng hệ thống hội thoại. Về bản chất, các mô hình này được huấn luyện trên lượng lớn dữ liệu văn bản nhằm học cách biểu diễn ngôn ngữ, dự đoán ngữ cảnh và sinh phản hồi phù hợp với yêu cầu của người dùng. Tuy nhiên, việc áp dụng trực tiếp mô hình ngôn ngữ lớn vào lĩnh vực y tế mà không có cơ chế kiểm soát có thể tạo ra nhiều rủi ro. Khuyết điểm chí mạng của LLM là chứng 'ảo giác' (hallucination) – mô hình có thể sinh ra thông tin không chính xác, suy diễn quá mức hoặc tạo ra các nội dung không có nguồn kiểm chứng rõ ràng. Đó chính là lý do hệ thống bắt buộc phải để ra thêm cơ chế truy xuất dữ liệu nội bộ và các chốt chặn an toàn để kiểm soát chặt chẽ đầu ra của AI.

2.3. Retrieval-Augmented Generation

Về bản chất, Retrieval-Augmented Generation là phương pháp kết hợp truy xuất thông tin với sinh ngôn ngữ tự nhiên, nhằm giúp mô hình trả lời dựa trên ngữ cảnh được cung

cấp từ nguồn dữ liệu bên ngoài. Thông thường, một pipeline RAG tiêu chuẩn sẽ đi qua các bước: nhận query, vector hóa (embedding), truy xuất tài liệu tương đồng (retrieval) và đưa các tài liệu liên quan vào prompt để mô hình sinh câu trả lời dựa trên ngữ cảnh đã truy xuất. Tuy nhiên, áp dụng lý thuyết này vào y tế không hề đơn giản. Nếu chỉ dùng thuật toán matching vector thông thường, khoảng cách ngữ nghĩa giữa các từ khóa y khoa rất dễ bị nhiễu. Do đó, hệ thống buộc phải được thiết kế thêm các bước tiền xử lý chuyên sâu: chuẩn hóa từ ngữ địa phương, tạo bộ lọc từ khóa dừng (stop-words) đối với các danh từ chung chung, và thiết lập một ngưỡng (threshold) nghiêm ngặt để loại bỏ các kết quả truy xuất có mức độ liên quan thấp trước khi chuyển sang bước sinh phản hồi trước khi đưa cho LLM xử lý.

2.4. Model Context Protocol

Model Context Protocol là giao thức giúp mô hình AI kết nối với các công cụ và nguồn dữ liệu bên ngoài theo cách có cấu trúc. Thay vì viết riêng từng kiểu tích hợp, MCP cho phép định nghĩa server công cụ, transport và schema kết quả. Ứng dụng chat có thể gọi MCP server để truy xuất dữ liệu, hiển thị UI tool result và mở rộng khả năng của mô hình.

Trong đồ án, MCP server đóng vai trò cầu nối giữa chatbot và các chức năng xử lý dữ liệu. Người dùng có thể đặt câu hỏi trong giao diện chat, hệ thống có thể kiểm tra dữ liệu nội bộ hoặc gọi công cụ tương ứng. Việc dùng MCP giúp kiến trúc mở hơn, dễ thêm công cụ mới như tìm hồ sơ bệnh án, đọc file, truy vấn dữ liệu hoặc gọi API y khoa.

2.5. Xử lý ảnh y tế

Ảnh y tế như X-quang, CT, MRI và DICOM có đặc điểm khác ảnh thông thường. Chúng thường là ảnh grayscale, có độ tương phản phụ thuộc thiết bị chụp, vùng quan tâm có thể nhỏ, nhiễu hoặc bị cắt. Một pipeline xử lý ảnh y tế thường gồm:

- Đọc ảnh và metadata.
- Chuyển ảnh về định dạng chuẩn.
- Chuẩn hóa kích thước.
- Chuẩn hóa cường độ sáng.
- Tăng tương phản nếu cần.

- Đánh giá chất lượng ảnh.
- Trích xuất đặc trưng hoặc đưa vào mô hình học sâu.
- Sinh kết quả dự đoán và cảnh báo an toàn.

Trong đồ án, backend sử dụng pipeline xử lý ảnh để tạo các thông tin như kích thước xử lý, định dạng, modality, độ tương phản, độ sắc nét, tỷ lệ vùng xương ước tính, mật độ cạnh và cảnh báo chất lượng. Các chỉ số này giúp người dùng hiểu vì sao model có hoặc không đưa ra kết luận.

2.6. Phân tích phim xương bằng mô hình AI

Trong phạm vi đề tài, bài toán phân tích phim xương được mô hình hóa dưới dạng bài toán phân loại ảnh. Ảnh X-quang hay CT sau khi upload sẽ được hệ thống tiền xử lý ảnh và đưa vào mô hình để dự đoán các nhân tương ứng như: bình thường, nghi gãy xương, hoặc thoái hóa. Thực tế thì giới nghiên cứu thường đi theo hai trường phái: một là dùng các model siêu nhẹ để trích xuất đặc trưng thủ công (như KNN), hai là xài luôn Deep Learning với các mạng CNN hay Vision Transformer. Ở góc độ của một sinh viên thực hiện đồ án, do không thể tiếp cận được nguồn dữ liệu X-quang khổng lồ có gán nhãn chuẩn y khoa, mình chọn chiến lược 'lấy ngắn nuôi dài'. Bản demo hiện tại đang chạy bằng một model trích xuất đặc trưng cực nhẹ chủ yếu để luồng pipeline không bị gãy. Dù vậy, hạ tầng backend thì đã được setup sẵn sàng để nhận các file .pt (TorchScript) từ những model CNN nặng đô hơn khi có đủ dữ liệu.

2.7. Phân tích xét nghiệm

Kết quả xét nghiệm máu và nước tiểu thường gồm tên chỉ số, giá trị, đơn vị, khoảng tham chiếu và trạng thái bất thường. Module xét nghiệm trong hệ thống thực hiện các bước:

- Nhận dữ liệu thủ công hoặc văn bản trích xuất từ file.
- Chuẩn hóa tên chỉ số.
- Xác định loại xét nghiệm: máu hoặc nước tiểu.
- So sánh giá trị với khoảng tham chiếu.

- Gán trạng thái: thấp, bình thường, cao, dương tính, âm tính, bất thường hoặc không rõ.
- Gán mức độ: thông tin, theo dõi, chú ý hoặc khẩn.
- Sinh tóm tắt và khuyến nghị theo dõi.

Mục tiêu của module xét nghiệm là giúp người dùng đọc nhanh kết quả, không thay thế kết luận của bác sĩ. Một chỉ số bất thường cần được diễn giải theo bối cảnh tuổi, giới, triệu chứng, bệnh nền, thuốc đang dùng và tiêu chuẩn của từng phòng xét nghiệm.

2.8. Fusion đa phương thức

Phần khai nhất khi làm module đọc xét nghiệm không phải là nhét data vào để so sánh, mà là bước chuẩn hóa dữ liệu đầu vào. Thực tế mỗi user lại nhập tên chỉ số theo một kiểu lộn xộn khác nhau. Đẩy mớ data này xuống backend mà không filter trước thì code tách ngay lập tức. Mình phải viết riêng một script để map (ánh xạ) toàn bộ tên chỉ số về format chuẩn. Chỉ khi dữ liệu đã sạch, mình mới cho móc nối vào logic dò tìm khoảng tham chiếu, từ đó hệ thống mới tự tin cắm cờ (flag) xem chỉ số đó đang ở ngưỡng cao, thấp hay bất thường một cách tron tru.

2.9. Đánh giá mô hình

Cầm một model có Accuracy lên tới 90%, nhiều lúc mình cũng tưởng là thành công rồi. Nhưng thực tế trong dataset y khoa, dữ liệu lớp 'normal' luôn áp đảo. Model chỉ cần nhắm mắt đoán bừa 'bình thường' cho mọi ca là đã đạt điểm Accuracy rất cao, trong khi lại bỏ sót sạch những ca gây xương nguy hiểm. Bị vấp ở điểm này, mình buộc phải gạt Accuracy sang một bên và thiết lập lại hệ thống đo lường, chuyển trọng tâm sang Macro-F1 để ép model phải dự đoán tốt đồng đều ở tất cả các nhãn, bất chấp việc dataset bị mất cân bằng trầm trọng.

2.10. Công nghệ sử dụng

Hệ thống sử dụng các công nghệ chính sau:

- Next.js: xây dựng giao diện và API route cho ứng dụng chat.
- React: xây dựng component UI.
- TypeScript: tăng độ an toàn kiểu dữ liệu ở frontend và API trung gian.

- Tailwind CSS: xây dựng giao diện responsive.
- shadcn/ui và Radix UI: cung cấp các component giao diện.
- Vercel AI SDK: tích hợp streaming response và các provider AI.
- Google Gemini/OpenAI/Anthropic/Groq: các provider mô hình ngôn ngữ.
- Model Context Protocol SDK: kết nối công cụ MCP.
- FastAPI: xây dựng backend xử lý y khoa.
- Pydantic: định nghĩa schema request/response.
- OpenCV/Numpy: xử lý ảnh và trích xuất đặc trưng.
- PostgreSQL và Drizzle ORM: lưu hội thoại trong ứng dụng Chat.
- SQLite: lưu tài khoản, upload history và prediction runs ở backend y khoa.
- Python scripts: huấn luyện model demo, chạy backend và đánh giá.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1. Bài toán đặt ra

Bài toán của đề án là xây dựng một hệ thống hỗ trợ y khoa có thể phục vụ hai nhóm nhu cầu. Nhóm thứ nhất là nhu cầu hỏi đáp thông tin bệnh lý bằng ngôn ngữ tự nhiên. Người dùng có thể hỏi "triệu chứng của bệnh gout", "dấu hiệu mang thai", "thoái hóa cột sống cổ có nguy hiểm không" hoặc các câu hỏi tương tự. Hệ thống cần hiểu đúng chủ đề, truy xuất dữ liệu liên quan và trả lời có cấu trúc.

Nhóm thứ hai là nhu cầu phân tích dữ liệu y khoa đầu vào gồm ảnh phim xương, xét nghiệm và thông tin lâm sàng. Người dùng có thể upload phim X-quang cổ tay, nhập vùng xương, nhập triệu chứng đau/sưng, upload xét nghiệm và yêu cầu hệ thống tổng hợp. Hệ thống cần trả về kết quả rõ ràng, có mức độ tin cậy, có giải thích và có khuyến cáo an toàn.

3.2. Tác nhân sử dụng hệ thống

Hệ thống hướng đến ba nhóm tác nhân:

- Người dùng thông thường/bệnh nhân: đặt câu hỏi, xem thông tin tham khảo, upload dữ liệu cá nhân để được hệ thống phân tích hỗ trợ.
- Nhân viên y tế/bác sĩ demo: dùng hệ thống để tham khảo nhanh, xem kết quả AI, so sánh nhiều nguồn dữ liệu và đánh giá mô hình.
- Quản trị/nhà phát triển: cấu hình model, API key, dataset, chạy server, kiểm thử và đánh giá.

Trong giao diện hiện tại, hệ thống có chức năng đăng nhập đơn giản cho phần y khoa, giúp gắn lịch sử upload và đánh giá với từng người dùng.

3.3. Yêu cầu chức năng

3.3.1. Chức năng chatbot

Hệ thống cần cho phép người dùng:

- Tạo hội thoại mới.
- Nhập câu hỏi tiếng Việt.

- Gửi câu hỏi và nhận câu trả lời dạng streaming.
- Lưu lịch sử hội thoại.
- Sử dụng model AI được cấu hình bằng API key.
- Truy xuất dữ liệu nội bộ khi câu hỏi liên quan đến bệnh lý.
- Tránh trả lời sai chủ đề khi câu hỏi chứa từ khóa chung.
- Hiện thị thông báo khi đang tìm kiếm và khi có lỗi.

Yêu cầu quan trọng của chatbot là hiểu ý định người dùng. Ví dụ, câu hỏi "triệu chứng của gout" phải ưu tiên dữ liệu gout, không được trả về cúm mùa chỉ vì trong bảng dữ liệu có các từ "triệu chứng". Câu hỏi "dấu hiệu mang thai" phải hiểu đây là dấu hiệu thai kỳ, không liên quan đến sốt xuất huyết. Do đó, hệ thống cần có lớp nhận diện chủ đề và fallback y khoa.

3.3.2. Chức năng phân tích phim xương

Người dùng có thể:

- Chọn file ảnh, PDF hoặc DICOM.
- Chọn loại phim: X-quang, CT, MRI hoặc không rõ.
- Nhập vùng xương như bàn tay, cổ tay, đầu gối, cột sống.
- Xem preview ảnh nếu là file ảnh.
- Gửi file đến backend để phân tích.
- Nhận kết quả metadata, chất lượng ảnh, đặc trưng ảnh và dự đoán model.
- Xem kết luận tham khảo như "nghi gãy xương" khi model đủ điều kiện.
- Xem cảnh báo "analysis only" nếu ảnh hoặc model chưa đủ tin cậy.
- Lưu lịch sử upload và so sánh hai lần phân tích.

3.3.3. Chức năng xét nghiệm

Người dùng có thể:

- Nhập tuổi, giới tính.
- Nhập chỉ số xét nghiệm máu/nước tiểu.
- Upload file xét nghiệm để trích xuất văn bản.

- Xem danh sách chỉ số, giá trị, đơn vị, khoảng tham chiếu.
- Xem trạng thái thấp/cao/bất thường/bình thường.
- Xem tóm tắt và bước tiếp theo khuyến nghị.
- Lưu lịch sử xét nghiệm và so sánh các lần upload.

3.3.4. Chức năng fusion

Người dùng có thể:

- Chọn lại ảnh/phim xương đã upload gần nhất.
- Nhập triệu chứng, tiền sử, điểm đau và vùng xương.
- Kết hợp dữ liệu xét nghiệm gần nhất.
- Chạy phân tích fusion.
- Xem fusion score, risk level, predicted label, confidence.
- Xem tín hiệu đóng góp từ ảnh, lâm sàng và xét nghiệm.
- Xem báo cáo có cấu trúc gồm bản chất/vị trí, mức độ, đánh giá tổng hợp và khuyến nghị.

3.3.5. Chức năng đánh giá mô hình

Người dùng có thể:

- Upload file CSV chứa nhãn thật và dự đoán.
- Tính accuracy, macro precision, macro recall, macro-F1, weighted-F1.
- Xem model tốt nhất theo macro-F1.
- Nhập thủ công một ca dự đoán gồm nhãn thật, dự đoán trước AI, image AI, clinical AI và multimodal.
- Lưu prediction run.
- Tải lại lịch sử upload ảnh/xét nghiệm để lấy kết quả vào phần đánh giá.

3.4. Yêu cầu phi chức năng

3.4.1. Tính dễ sử dụng

Khi làm frontend, một trong những điều tối kỵ là nhồi nhét quá nhiều tính năng vào một màn hình. Với đặc thù của một hệ thống y khoa đa luồng (vừa tương tác chat, vừa phân tích ảnh X-quang, vừa đọc xét nghiệm), yêu cầu sống còn là UI không được làm người dùng quá tải thông tin. Thay vì một giao diện phẳng, hệ thống được thiết kế theo layout điều hướng một chạm với sidebar cố định, bóc tách hoàn toàn các module (New Chat, Upload phim xương, Xét nghiệm, Fusion, Đánh giá) thành các route riêng biệt trong Next.js. Quyết định kiến trúc này không chỉ giúp người dùng có không gian làm việc 'dễ thở' hơn, mà ở góc độ quản lý code, nó ngăn chặn tình trạng phình to global state và giảm thiểu tối đa các pha re-render vô nghĩa của React khi người dùng chuyển đổi qua lại giữa các tác vụ.

3.4.2. Tính phản hồi

Trong các ứng dụng tích hợp AI, bài toán khó chịu nhất luôn là độ trễ (latency). Việc phải chờ backend FastAPI chạy xong pipeline xử lý ảnh hoặc LLM sinh xong toàn bộ câu trả lời rất dễ khiến người dùng hiểu nhầm rằng hệ thống đã bị treo (crash). Để tối ưu hóa hiệu năng cảm nhận (perceived performance), tính phản hồi phải được xử lý triệt để bằng kỹ thuật. Đối với module chatbot, luồng dữ liệu bắt buộc phải chạy qua cơ chế streaming: LLM nhả text đến đâu, frontend render ngay lập tức đến đó. Với các tác vụ nặng như upload và phân tích phim, hệ thống được bọc bằng các bộ loading skeleton kết hợp cơ chế bắt lỗi (error handling) chi tiết, đảm bảo giao diện luôn phản hồi trạng thái minh bạch để người dùng biết hệ thống đang xử lý tác vụ ngầm.

3.4.3. Tính mở rộng

Một hệ thống AI không thể được code theo kiểu 'hard-code' chết cứng. Để dự án có vòng đời dài hơn một bản demo đồ án, kiến trúc phải tuân thủ nguyên tắc tách ghép lỏng lẻo (loose coupling). Lớp giao diện (Frontend) và logic AI (Backend) được bóc tách hoàn toàn, giao tiếp qua các API tuân thủ nghiêm ngặt chuẩn dữ liệu. Nhờ vậy, hệ thống đạt được tính 'plug-and-play' (cắm và chạy). Sau này, nếu muốn chuyển đổi provider LLM (từ Google Gemini sang OpenAI hay Anthropic), đập bỏ bộ model ảnh cũ để cắm mạng

CNN mới, hoặc nhúng thêm các MCP tool, người phát triển chỉ cần khai báo lại biến môi trường và config ở backend mà không phải phá vỡ hay viết lại bất kỳ component UI nào.

3.4.4. Tính an toàn

Làm sản phẩm y tế, ám ảnh lớn nhất là việc người dùng tin tưởng mù quáng vào kết quả của máy móc. Vì vậy, từ UI cho tới logic backend, nguyên tắc sống còn được thiết lập là: AI chỉ đóng vai trò trợ lý, tuyệt đối không được 'vượt quyền' bác sĩ. Hệ thống được cài cắm các chốt chặn cứng: bất cứ khi nào model trả về độ tin cậy thấp, hệ thống tự động ngắt tính năng phán đoán, fallback về trạng thái 'Analysis Only' và block luôn kết quả y khoa. Đồng thời, các cảnh báo đỏ yêu cầu người dùng tới bệnh viện được hard-code sẵn nếu hệ thống parse (trích xuất) ra được các keyword lâm sàng mang tính chất nguy hiểm, đảm bảo ranh giới an toàn y khoa.

3.4.5. Tính kiểm thử

Backend cần có test cho các module xử lý ảnh, xét nghiệm, lâm sàng, fusion và đánh giá. Frontend cần đảm bảo TypeScript compile và API route hoạt động.

3.5. Use case tổng quát

3.5.1. Use case hỏi bệnh lý

Người dùng nhập câu hỏi trong chat. Hệ thống nhận câu hỏi, xác định đây là câu hỏi y khoa, nhận diện chủ đề, truy xuất dữ liệu nội bộ nếu có, gọi model AI hoặc trả lời fallback, sau đó hiển thị kết quả. Nếu dữ liệu nội bộ không liên quan, hệ thống không ép dùng kết quả sai mà trả lời theo kiến thức tổng quát có cảnh báo.

3.5.2. Use case phân tích phim xương

Người dùng đăng nhập, upload phim, chọn modality và nhập vùng xương. Frontend gửi file đến API route của Chat. API route chuyển tiếp request đến FastAPI backend. Backend đọc file, tiền xử lý ảnh, đánh giá chất lượng, trích xuất đặc trưng, gọi model, lưu upload history nếu có token, trả response về frontend. Frontend hiển thị kết quả và kết luận tham khảo.

3.5.3. Use case phân tích xét nghiệm

Người dùng nhập hoặc upload kết quả xét nghiệm. Backend chuẩn hóa chỉ số, so sánh với khoảng tham chiếu và trả về danh sách bất thường. Frontend hiển thị tổng quan, số chỉ số bất thường, chỉ số khẩn và khuyến nghị.

3.5.4. Use case fusion

Người dùng chọn dữ liệu ảnh, nhập lâm sàng và xét nghiệm. Backend chạy lại phân tích ảnh nếu cần, phân tích lâm sàng, phân tích xét nghiệm, sau đó tính fusion score. Kết quả được hiển thị dưới dạng báo cáo tổng hợp.

3.5.5. Use case đánh giá

Người dùng chuẩn bị file CSV chứa các cột nhãn thật, dự đoán và model name. Hệ thống tính metrics cho từng model, so sánh macro-F1 và hiển thị kết quả. Người dùng có thể dùng kết quả này trong báo cáo thực nghiệm.

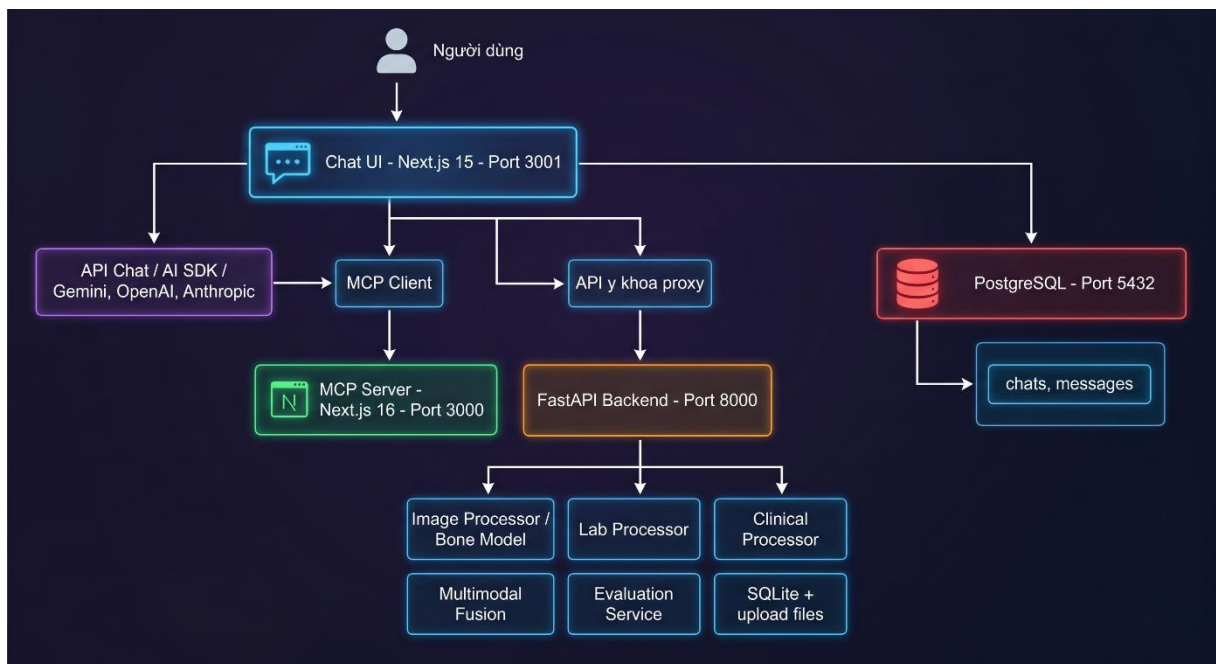
CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU

4.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc nhiều lớp:

- Lớp giao diện: ứng dụng *Chat* xây dựng bằng Next.js, React và TypeScript.
- Lớp API trung gian: API routes trong Next.js nhận request từ frontend và gọi backend tương ứng.
- Lớp MCP: *mcp-server* cung cấp công cụ theo chuẩn Model Context Protocol.
- Lớp xử lý AI/y khoa: FastAPI backend trong thư mục *mcp-server/app*.
- Lớp dữ liệu: PostgreSQL cho hội thoại Chat, SQLite cho dữ liệu y khoa backend, file system cho upload và model.

Sơ đồ triển khai tổng quát:



Hình 4.1: Sơ đồ kiến trúc tổng thể hệ thống.

4.2. Thiết kế frontend

Để đảm bảo UI/UX mượt mà khi người dùng tương tác liên tục với nhiều module y khoa cùng lúc, toàn bộ lớp giao diện được build trên nền Next.js App Router kết hợp React

19. Khác với các project thông thường, hệ thống này đòi hỏi strict type cực cao ở các API trung gian nhận kết quả phân tích y tế, nên TypeScript là lựa chọn bắt buộc ngay từ đầu. Giao diện được dựng hoàn toàn bằng Tailwind CSS, kết hợp shadcn/ui để tối ưu thời gian lên component mà vẫn giữ được tính nhất quán. Việc bóc tách riêng từng màn hình (như Chat, Upload phim xương, Fusion) giúp state không bị phình to và hạn chế tình trạng re-render không đáng có khi user chuyển đổi qua lại giữa các luồng tác vụ.

4.3. Thiết kế backend y khoa

Backend y khoa được đặt trong *mcp-server/app* và xây dựng bằng FastAPI. Các module chính gồm:

- *main.py*: khai báo ứng dụng FastAPI, route API, xác thực và lưu upload.
- *schemas.py*: định nghĩa schema Pydantic cho request/response.
- *processor.py*: xử lý ảnh, tiền xử lý, quality metrics và feature extraction.
- *model.py*: load model và suy luận.
- *clinical.py*: phân tích dữ liệu lâm sàng.
- *lab_results.py*: phân tích xét nghiệm.
- *lab_file_parser.py*: trích xuất văn bản xét nghiệm từ file.
- *multimodal.py*: fusion ảnh, lâm sàng và xét nghiệm.
- *evaluation.py*: tính metrics đánh giá model.
- *storage.py*: lưu dữ liệu người dùng, upload, bệnh nhân và prediction run.
- *security.py*: hash password, JWT token và xác thực.

Backend dùng Pydantic để đảm bảo dữ liệu đầu vào/đầu ra có cấu trúc. Điều này giúp frontend dễ hiển thị kết quả và giúp test dễ viết hơn.

4.4. Thiết kế cơ sở dữ liệu Chat

Ứng dụng Chat sử dụng PostgreSQL và Drizzle ORM. Schema chính gồm bảng *chats* và *messages*.

Bảng *chats*:

Trường	Kiểu	Ý nghĩa
--------	------	---------

id	text	Khóa chính, tạo bằng nanoid
user_id	text	Định danh người dùng
title	text	Tiêu đề hội thoại
created_at	timestamp	Thời điểm tạo
updated_at	timestamp	Thời điểm cập nhật

Bảng *messages*:

Trường	Kiểu	Ý nghĩa
id	text	Khóa chính
chat_id	text	Khóa ngoại đến bảng chats
role	text	user, assistant hoặc tool
parts	json	Nội dung tin nhắn dạng JSON
created_at	timestamp	Thời điểm tạo

Việc lưu *parts* dưới dạng JSON giúp hệ thống hỗ trợ nhiều loại nội dung như text, tool call, tool result và attachment.

4.5. Thiết kế dữ liệu backend y khoa

Backend y khoa dùng SQLite để lưu các thông tin phục vụ demo và đánh giá:

- Tài khoản người dùng.
- Bệnh nhân.
- Hồ sơ lâm sàng.
- Upload phim xương/xét nghiệm.
- Prediction run dùng cho đánh giá.

Mỗi upload lưu các thông tin như:

- Người sở hữu.
- Loại upload: image hoặc lab.
- Tên file.
- Content type.

- Đường dẫn file lưu trên server.
- Kích thước file.
- Modality.
- Vùng xương.
- Source text nếu là xét nghiệm.
- Kết quả phân tích dạng JSON.
- Trạng thái nhãn và khả năng dùng cho training.
- Thời điểm tạo.

Thiết kế này giúp hệ thống có thể mở rộng thành kho dữ liệu huấn luyện trong tương lai. Khi người dùng hoặc chuyên gia xác nhận nhãn đúng, upload có thể được đánh dấu là usable for training.

4.6. Thiết kế API

Các API chính của backend FastAPI gồm:

- *GET /health*: kiểm tra trạng thái backend và model.
- *POST /api/auth/register*: đăng ký tài khoản.
- *POST /api/auth/login*: đăng nhập.
- *POST /api/images/analyze*: phân tích phim xương.
- *POST /api/labs/analyze*: phân tích xét nghiệm từ dữ liệu nhập.
- *POST /api/labs/analyze-file*: phân tích xét nghiệm từ file.
- *POST /api/clinical/analyze*: phân tích dữ liệu lâm sàng.
- *POST /api/multimodal/analyze*: phân tích fusion.
- *POST /api/evaluation/evaluate*: đánh giá model từ records.
- *POST /api/evaluation/metrics-file*: đánh giá model từ CSV.
- *GET /api/uploads*: lấy lịch sử upload.
- *GET /api/uploads/{id}/file*: mở file upload.
- *POST /api/evaluation/runs*: lưu một prediction run.

Ở phía Chat Next.js, các route proxy tương ứng được đặt trong *Chat/app/api*, ví dụ:

- *api/medical-images/analyze*
- *api/labs/analyze*
- *api/labs/analyze-file*
- *api/multimodal/analyze*
- *api/evaluation/metrics-file*
- *api/evaluation/runs*
- *api/uploads*

Proxy giúp frontend không cần gọi trực tiếp backend port 8000, đồng thời có thể thêm header xác thực và xử lý lỗi tập trung.

4.7. Thiết kế luồng chatbot RAG/MCP

Luồng chatbot gồm các bước:

- Người dùng nhập câu hỏi.
- Frontend gửi message đến *POST /api/chat*.
- API route xác định nội dung câu hỏi cuối cùng của người dùng.
- Hệ thống kiểm tra câu hỏi có thuộc nhóm y khoa hay không.
- Nếu là câu hỏi y khoa, hệ thống nhận diện chủ đề như gout, thoái hóa cột sống cổ, mang thai.
- Hệ thống tìm dữ liệu nội bộ liên quan hoặc dùng fallback y khoa nếu dữ liệu nội bộ không phù hợp.
- Nếu cần, hệ thống dùng AI SDK để gọi mô hình ngôn ngữ lớn.
- Kết quả được stream về frontend.
- Tin nhắn được lưu vào database.

Một trong những rủi ro lớn nhất khi làm RAG cho y tế là ảo giác từ khóa. Quá trình test ban đầu trả về kết quả rất ngớ ngẩn: user hỏi 'dấu hiệu mang thai', hệ thống lại hồn nhiên nhại nguyên một đoạn về bệnh 'sốt xuất huyết' đắp vào prompt, đơn giản vì hai vector đều chứa từ 'dấu hiệu'. Để xử lý triệt để bug logic này, mình phải viết thêm một lớp filter cứng nhận diện chủ đề y khoa ngay trước bước truy xuất. Lúc này, các từ chung chung như 'triệu chứng' hay 'dấu hiệu' bị đẩy thẳng vào danh sách stop words, ép thuật toán chỉ được matching dựa trên những keyword mang tính định danh bệnh lý thực sự..

4.8. Thiết kế luồng phân tích ảnh

Luồng phân tích ảnh:

- Người dùng upload file.
- Frontend kiểm tra định dạng và dung lượng.
- Frontend gửi multipart form-data đến API route.
- API route chuyển tiếp đến FastAPI.
- Backend đọc ảnh bằng module *image_io*.
- Backend tiền xử lý ảnh bằng *MedicalImageProcessor*.
- Backend tính quality metrics.
- Backend trích xuất feature vector.
- Backend gọi *BoneModelService* để dự đoán.
- Backend lưu upload nếu có token người dùng.
- Backend trả JSON response về frontend.
- Frontend hiển thị kết quả.

Kết quả ảnh gồm bốn nhóm:

- Metadata: filename, modality, width, height, source format.
- Preprocessing: normalized, contrast enhanced, resized size, original size.
- Quality: mean intensity, contrast, sharpness, dynamic range, dark/bright ratio, warnings.
- Prediction: status, labels, probabilities, top_label, confidence, note.

4.9. Thiết kế quy tắc an toàn cho model ảnh

Model ảnh có hai trạng thái kết quả:

- *model_prediction*: model đủ điều kiện để đưa ra gợi ý nhãn.
- *analysis_only*: chỉ hiển thị phân tích hỗ trợ, không đưa ra kết luận bệnh lý cụ thể.

Lúc code phần phân loại, mình bị kẹt giữa hai thái cực cực kỳ đau đầu. Ban đầu, thả lỏng một chút là model phán 'gãy xương' với độ tự tin 100% cho hàng loạt ảnh, nhìn vào rất 'giả trân'. Bực mình, mình siết rule lại, thêm hàng loạt điều kiện chặn cứng. Kết quả là hệ thống trở nên nhát cáy, ảnh gãy rõ rành rành cũng chỉ dám nhả về trạng thái

'analysis only', khiến model AI mất đi giá trị cốt lõi. Giải pháp cuối cùng là phải đề ra một bộ rule lai: chỉ khi xác suất thô đạt đỉnh, margin giữa các top label đủ rộng và feature vector nằm gọn trong vùng an toàn của tập train, hệ thống mới được phép bật cờ chốt hạ kết quả.

Vì vậy hệ thống được hiệu chỉnh theo hướng có quy tắc rõ:

- Nếu model có bằng chứng mạnh cho gãy xương, ví dụ xác suất thô rất cao, margin lớn và feature gần vùng training, hệ thống cho phép hiển thị "ngghi gãy xương".
- Confidence được hiệu chỉnh, không dùng trực tiếp xác suất KNN thô.
- Nếu ảnh quá tối, quá sáng, nhiễu hoặc không giống dữ liệu training, hệ thống chuyển sang analysis only.
- Nếu model demo không đủ độ tin cậy cho các nhãn khác, hệ thống không khẳng định.
- Luôn hiển thị cảnh báo rằng kết quả chỉ hỗ trợ sàng lọc, cần bác sĩ đối chiếu phim gốc.

Quy tắc này giúp cân bằng giữa lợi ích của AI và yêu cầu an toàn.

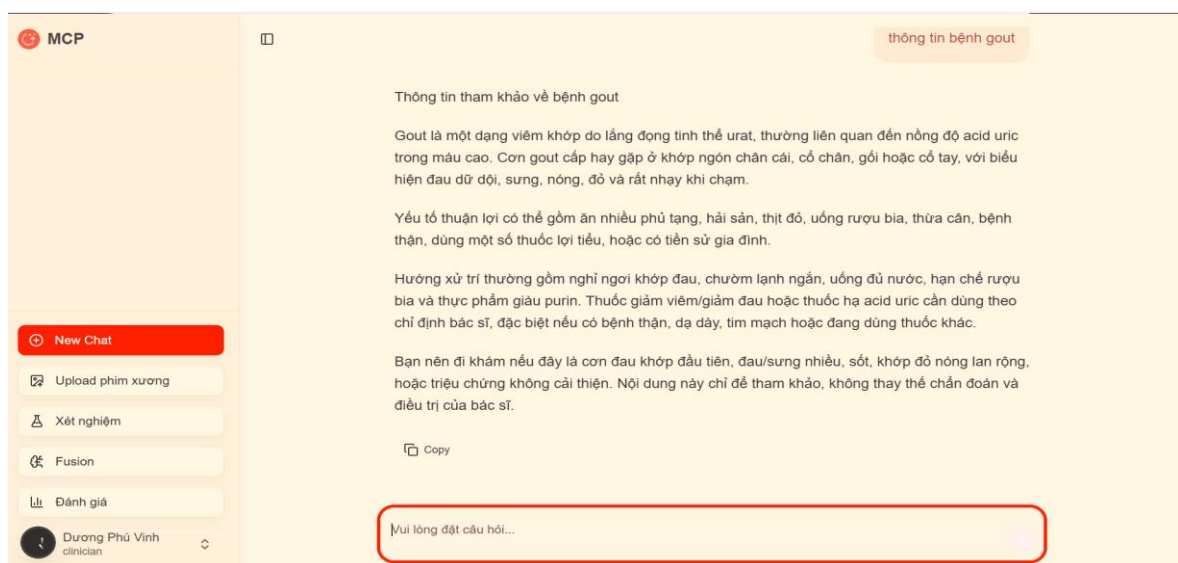
CHƯƠNG 5. XÂY DỰNG CÁC MODULE CHỨC NĂNG

5.1. Module giao diện chatbot

Module chatbot là màn hình trung tâm của hệ thống. Người dùng có thể tạo hội thoại mới, nhập câu hỏi và xem phản hồi từ AI. Giao diện sử dụng layout gồm sidebar bên trái và vùng chat chính bên phải. Sidebar có các nút điều hướng đến các module khác như upload phim xương, xét nghiệm, fusion và đánh giá.

Tin nhắn người dùng được hiển thị ở phía phải, tin nhắn assistant hiển thị ở vùng nội dung chính. Khi người dùng gửi câu hỏi, hệ thống hiển thị trạng thái đang tìm kiếm/tổng hợp thông tin. Cơ chế streaming giúp người dùng thấy phản hồi dần thay vì chờ toàn bộ câu trả lời.

Trong quá trình tích hợp cơ chế streaming với Next.js, một vấn đề khá khó chịu phát sinh: request gửi đi, UI đã hiện trạng thái loading nhưng stream lại không kết thúc đúng cách. Hậu quả là toàn bộ giao diện chat bị treo cứng. Sau khi cắm mặt vào VS Code để debug luồng dữ liệu, nguyên nhân được xác định là do cách handle stream response ở API route chưa chuẩn. Để giải quyết triệt để bài toán trải nghiệm này, mình đã cấu trúc lại route chat, đồng thời bổ sung thêm tính năng 'Stop generating' ngay trên frontend để người dùng có thể chủ động ngắt kết nối khi cần.



Hình 5.1: Giao diện chatbot khi người dùng hỏi thông tin bệnh lý.

5.2. Module nhận diện câu hỏi y khoa

Để chatbot trả lời đúng, hệ thống cần hiểu ý định và chủ đề câu hỏi. Module nhận diện câu hỏi y khoa sử dụng danh sách từ khóa và alias tiếng Việt không dấu/có dấu. Ví dụ:

- Gout: gout, gut, thống phong, acid uric.
- Thoái hóa cột sống cổ: thoái hóa cổ, cột sống cổ, cervical spondylosis.
- Mang thai: mang thai, có thai, thai kỳ, thai nghén, dấu hiệu có bầu.
- Tiểu đường: đái tháo đường, tiểu đường, glucose, HbA1c.
- Cúm mùa: cúm, influenza.
- Sốt xuất huyết: dengue, xuất huyết.

Khi phát hiện chủ đề, hệ thống chỉ nhận các dữ liệu nội bộ cùng chủ đề. Các từ chung như "triệu chứng", "dấu hiệu", "bệnh", "tìm thông tin" được đưa vào stop words để tránh ảnh hưởng điểm tìm kiếm.

Ví dụ, với câu hỏi "triệu chứng của gout", hệ thống ưu tiên trả lời về cơn đau khớp dữ dội, sưng nóng đỏ, thường gặp ở khớp ngón chân cái, tái phát từng đợt và liên quan acid uric. Hệ thống không lấy dữ liệu cúm mùa dù trong dữ liệu cúm có từ "triệu chứng".

5.3. Module truy xuất dữ liệu nội bộ

Dữ liệu nội bộ được đọc từ file bảng tính trong thư mục *tool*. Module truy xuất thực hiện các bước:

- Đọc file Excel.
- Chuẩn hóa chữ tiếng Việt.
- Tách từ khóa câu hỏi.
- Tính điểm liên quan giữa câu hỏi và từng dòng dữ liệu.
- Lọc theo chủ đề nếu có chủ đề y khoa rõ ràng.
- Chọn các dòng có điểm cao nhất.
- Trả ngữ cảnh cho chatbot.

Trong phiên bản ban đầu, hệ thống dễ bị nhiễu bởi các từ khóa chung. Sau khi bổ sung nhận diện chủ đề, độ chính xác truy xuất được cải thiện đáng kể trong các tình huống hỏi cụ thể như gout, thoái hóa cột sống cổ hoặc mang thai.

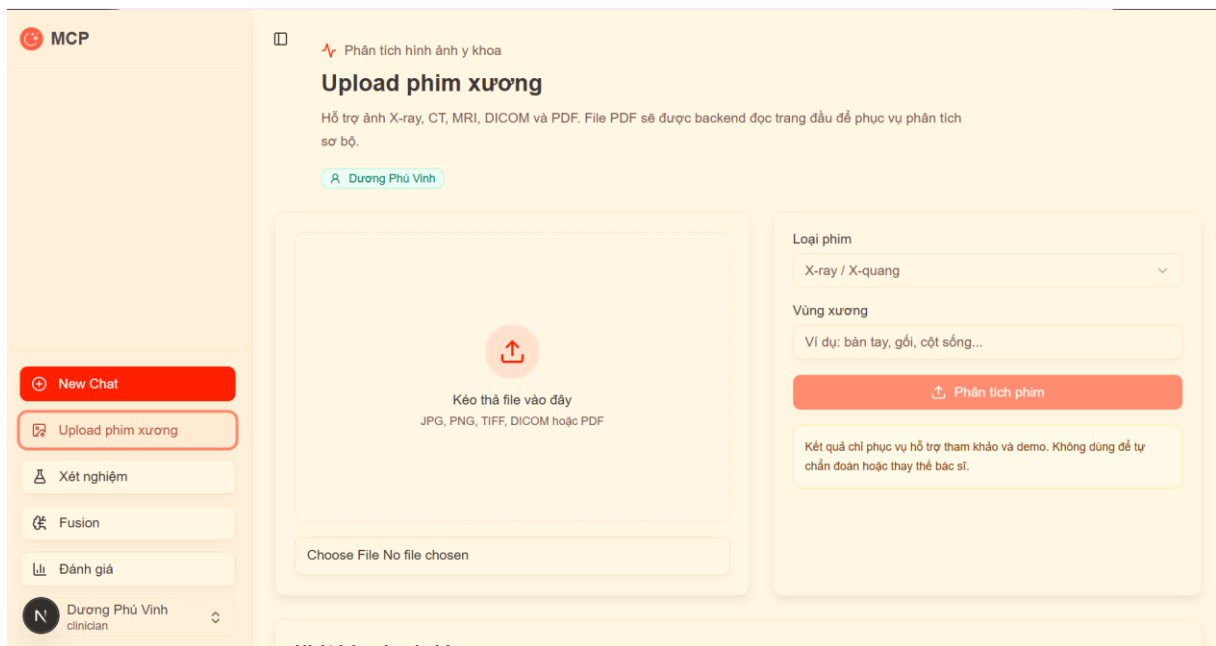
5.4. Module upload phim xương

Module upload phim xương cho phép người dùng chọn file phim và nhập thông tin cơ bản. Frontend kiểm tra định dạng được hỗ trợ gồm JPG, PNG, BMP, TIFF, DICOM và PDF. Dung lượng tối đa được giới hạn để tránh tải file quá lớn.

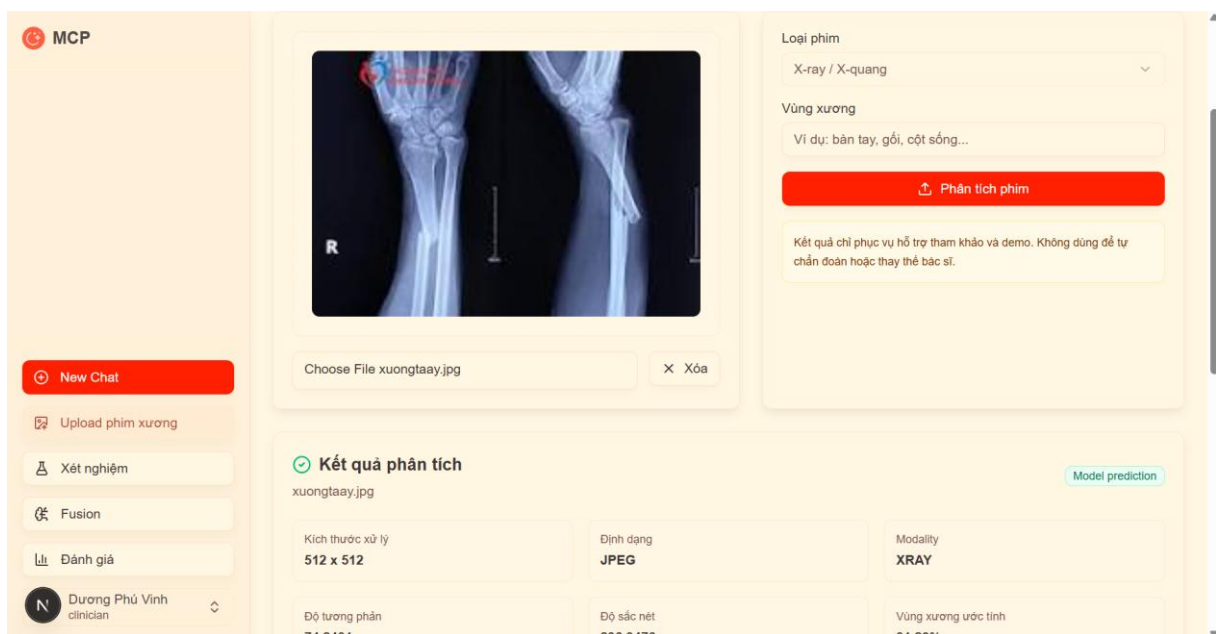
Sau khi upload, hệ thống hiển thị:

- Tên file.
- Kích thước xử lý.
- Định dạng.
- Modality.
- Độ tương phản.
- Độ sắc nét.
- Vùng xương ước tính.
- Kết luận tham khảo.
- Độ tin cậy nếu có model prediction.
- Các cảnh báo chất lượng.

Kết luận được trình bày dưới dạng "Model gợi ý" thay vì "Chẩn đoán", nhằm nhấn mạnh vai trò hỗ trợ.



Hình 5.2 Giao diện upload phim xương.



Hình 5.3: Kết quả phân tích ảnh X-quang nghi gãy xương.

5.5. Pipeline xử lý ảnh

Tuyệt đối không được ném thẳng ảnh người dùng upload vào thẳng model. Thực tế test cho thấy, ảnh X-quang mỗi người chụp một kiểu, cái thì tối thui, cái thì cháy sáng. Nếu cứ thế feed vào mạng, kết quả rác (garbage-in, garbage-out) là chắc chắn. Do đó, mình phải cài thêm một pipeline tiền xử lý khá 'khoai' bằng OpenCV. Bắt buộc phải ép toàn bộ ảnh về grayscale, chuẩn hóa lại cường độ pixel và kéo lại độ tương phản cho đồng

đều. Quan trọng nhất là khâu check quality: nếu module tính toán phát hiện ảnh quá tối hoặc mất nét, hệ thống sẽ thẳng tay block luôn từ vòng gửi xe để tránh model học lệch và đưa ra dự đoán sai.

5.6. Module model ảnh

Module model ảnh được cài đặt trong *BoneModelService*. Service có nhiệm vụ load model từ cấu hình, kiểm tra model sẵn sàng và dự đoán trên ảnh đã xử lý.

Hệ thống hỗ trợ hai loại model:

- Feature model dạng JSON: dùng đặc trưng ảnh và thuật toán nhẹ như KNN/centroid.
- TorchScript model: dùng CNN được huấn luyện bằng PyTorch và export ra file *.pt*.

Trong demo, feature model được dùng để minh họa lợi ích của model AI train. Model không chỉ trả nhãn, mà còn trả probabilities, top label, confidence và note. Confidence được hiệu chỉnh bằng vote margin và khoảng cách đặc trưng để tránh trường hợp xác suất thô 100% nhưng không đáng tin.

Với ảnh gãy xương rõ và model có bằng chứng mạnh, hệ thống hiển thị "nghi gãy xương" với độ tin cậy cao. Với ảnh không phải phim xương hoặc ảnh chất lượng kém, hệ thống chuyển sang analysis only.

5.7. Module xét nghiệm

Module xét nghiệm cho phép nhập dữ liệu theo hai cách: nhập từng dòng chỉ số hoặc upload file xét nghiệm. Frontend hỗ trợ danh sách chỉ số thường gặp như WBC, RBC, HGB, HCT, PLT, Glucose, Creatinine, Urea, ALT, AST, CRP, Protein niệu, Ketone, Blood, Leukocyte, Nitrite, pH và SG.

Backend phân tích từng chỉ số:

- Chuẩn hóa tên chỉ số.
- Xác định nhóm xét nghiệm.
- So sánh với khoảng tham chiếu.
- Xác định trạng thái.

- Xác định mức độ.
- Viết diễn giải ngắn.
- Đếm số chỉ số bất thường và số chỉ số khẩn.

Kết quả được trả về gồm danh sách item, summary, abnormal count, urgent count, recommended next steps và safety note.

Hình 5.4: Giao diện nhập xét nghiệm.

Hình 5.5: Kết quả phân tích xét nghiệm.

5.8. Module lâm sàng

Module lâm sàng nhận thông tin tuổi, giới, triệu chứng, tiền sử và chỉ số lâm sàng. Hệ thống chuẩn hóa dữ liệu và tạo các risk item. Ví dụ, đau nhiều, sưng nề, biến dạng sau chấn thương hoặc hạn chế vận động có thể làm tăng mức ưu tiên xem phim.

Module lâm sàng không cố gắng chẩn đoán bệnh độc lập, mà cung cấp tín hiệu cho fusion. Điều này phù hợp với thực tế y khoa: ảnh, triệu chứng và xét nghiệm cần được đọc trong cùng bối cảnh.

5.9. Module fusion đa phương thức

Ban đầu, việc gộp chung kết quả từ ảnh, lâm sàng và xét nghiệm khá lúng túng vì mỗi nguồn có một độ nhiễu khác nhau. Thay vì cộng trung bình đơn thuần, mình quyết định áp dụng late fusion có trọng số. Các con số 0.46 (ảnh), 0.34 (lâm sàng) và 0.20 (xét nghiệm) không phải ngẫu nhiên mà ra. Qua nhiều vòng test thử với các ca gãy kín đáo, mình nhận thấy ảnh X-quang mang tính quyết định cao nhất nên phải gán trọng số lớn nhất (0.46). Tuy nhiên, nếu bỏ qua hoàn toàn triệu chứng đau thực tế của bệnh nhân thì model lại rất dễ đánh giá sai mức độ nghiêm trọng. Do đó, điểm lâm sàng được neo ở mức 0.34 để làm 'chốt chặn' đối chiếu.

The screenshot shows the 'MCP Multimodal fusion' interface. The main title is 'Phân tích ảnh + lâm sàng'. Below it, a subtitle reads: 'Pipeline hợp nhất đặc trưng ảnh y khoa và dữ liệu lâm sàng bằng late fusion có trọng số.' A user profile for 'Dương Phú Vinh' is displayed. A status bar shows: 'Phim mới nhất: xuongtaay.jpg (20:19 21/5/26)', 'Xét nghiệm mới nhất: 20:23 21/5/26', and 'Khám lần trước: chưa có'. The form is divided into two main sections. The left section, titled 'Ảnh/phim xương', includes a file upload area (currently showing 'Choose File No file chosen'), a selected file 'Đã chọn: xuongtaay.jpg', a dropdown for 'Loại phim' (set to 'X-ray'), a dropdown for 'Vùng xương' (set to 'gối, tay, cột sống...'), a 'Tuổi' field (47), a 'Giới tính' dropdown (set to 'Nam'), a 'Triệu chứng' text area (containing 'Mỗi dòng một triệu chứng', 'đau khớp', 'sưng sau té ngã'), and a 'Tiền sử bệnh' field. The right section is a large box with a heart rate icon and the text 'Chưa có kết quả fusion'.

Hình 5.6: Giao diện fusion đa phương thức.

The screenshot displays the MCP (Medical Case Platform) interface for a structured report. The top bar shows the MCP logo and three tabs: 'Phim mới nhất: xuongtaay.jpg (20:19 21/5/26)', 'Xét nghiệm mới nhất: 20:23 21/5/26', and 'Khám lần trước: chưa có'. The left sidebar contains a 'New Chat' button and a list of actions: 'Upload phim xương', 'Xét nghiệm', 'Fusion', and 'Đánh giá'. The main content area is divided into several sections:

- Ảnh/phim xương:** A file upload section showing 'Choose File No file chosen' and 'Đã chọn: xuongtaay.jpg'.
- Loại phim:** A dropdown menu set to 'X-ray'.
- Vùng xương:** A dropdown menu set to 'gối, tay, cột sống...'.
- Tuổi:** A text input field containing '47'.
- Giới tính:** A dropdown menu set to 'Nam'.
- Triệu chứng:** A text area containing 'Mỗi dòng một triệu chứng đau khớp sưng sau té ngã'.
- Tiền sử bệnh:** A text area containing 'loãng xương viêm khớp'.
- Điểm đau:** A text input field containing '0-10'.
- Chỉ số xét nghiệm máu/nước tiểu:** A text input field.

On the right side, there is a section titled 'Nguy cơ thấp' with a warning: 'Chưa thấy tín hiệu bất thường nổi bật'. Below this, a table shows the 'Điểm fusion' (0.3335) and 'Độ tin cậy' (66.6%). A section titled 'KẾT LUẬN THAM KHẢO' contains three points:

- 1. Bản chất và vị trí tổn thương:** Trên X-quang vùng vùng xương đã chụp, model ảnh gợi ý nghi gãy xương với độ tin cậy 95.0%. Cần xác định lại vị trí chính xác trên phim gốc, đường vỏ xương, khe khớp, trục xương và mô mềm lân cận.
- 2. Mức độ:** Mức độ thấp: fusion score 0.33. Độ tin cậy model ảnh 95.0%. Chưa thấy tín hiệu nguy cơ nổi bật trong dữ liệu đã nhập.
- 3. Đánh giá toàn diện và khuyến nghị:** Kết quả fusion ở mức thấp với điểm 0.33. Yếu tố lâm sàng đáng chú ý: Chưa ghi nhận dấu hiệu nguy cơ rõ từ dữ liệu.

Hình 5.7: Báo cáo structured report sau fusion.

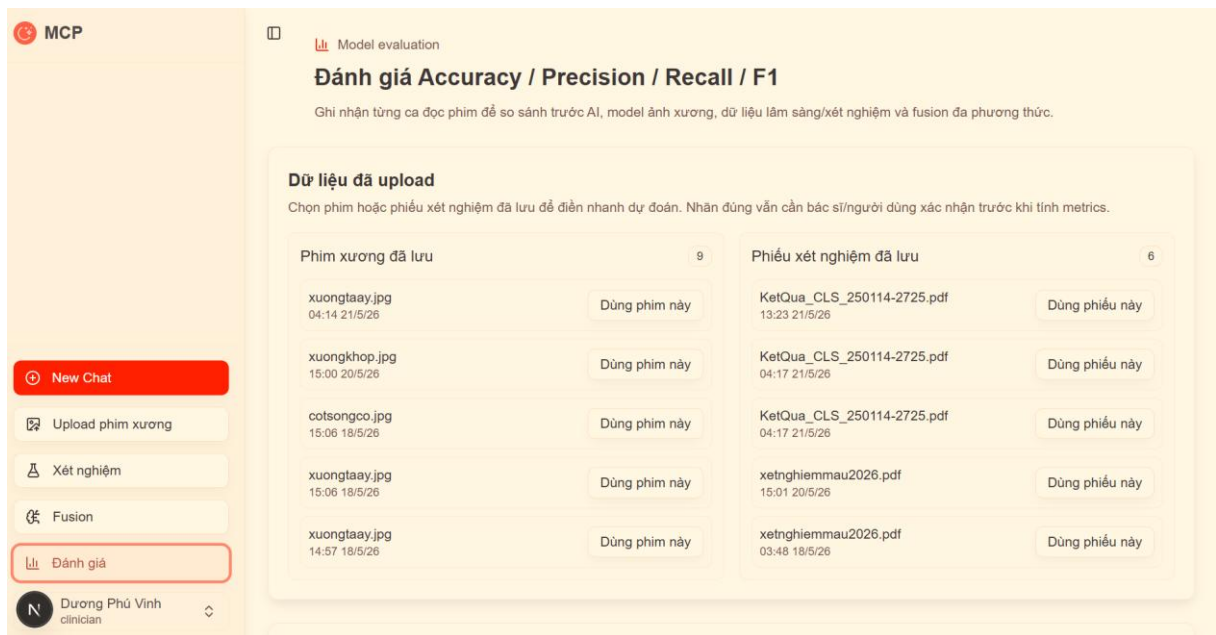
5.10. Module đánh giá mô hình

Module đánh giá cho phép nhập kết quả dự đoán của nhiều model. Mỗi record gồm nhãn thật, nhãn dự đoán và tên model. Hệ thống nhóm record theo model, tính confusion matrix và các chỉ số.

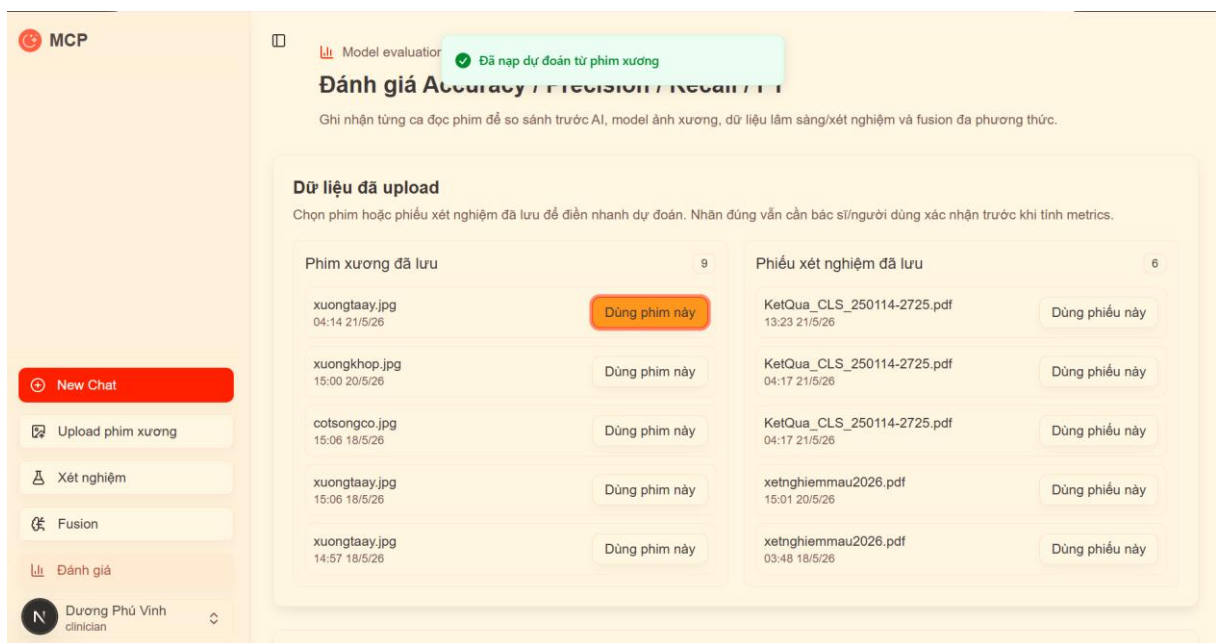
Ví dụ, có thể so sánh:

- *before_ai*: đánh giá trước khi dùng AI.
- *image_ai*: model chỉ dùng ảnh.
- *clinical_ai*: model chỉ dùng lâm sàng.
- *multimodal*: model hoặc rule fusion kết hợp nhiều nguồn.

Việc so sánh này giúp chứng minh lợi ích của model AI train. Nếu multimodal có macro-F1 cao hơn image-only hoặc clinical-only, báo cáo có cơ sở để kết luận fusion giúp cải thiện kết quả.



Hình 5.8: Giao diện đánh giá model.



Hình 5.9: Bảng metrics so sánh các model.

5.11. Module xác thực và lịch sử

Phân y khoa có chức năng đăng ký, đăng nhập và lưu token. Sau khi đăng nhập, người dùng có thể lưu lịch sử upload ảnh, xét nghiệm và prediction run. Điều này giúp hệ thống có trải nghiệm liên tục: ảnh đã upload ở module phim xương có thể được dùng lại ở module fusion; kết quả xét nghiệm gần nhất cũng có thể tự động nạp vào fusion.

Trong phạm vi demo, cơ chế xác thực dùng JWT và hash password. Khi phát triển thực tế, cần bổ sung các yêu cầu bảo mật cao hơn như HTTPS, phân quyền chi tiết, audit log, mã hóa dữ liệu nhạy cảm và tuân thủ quy định bảo vệ dữ liệu y tế.

CHƯƠNG 6. HUẤN LUYỆN, SUY LUẬN VÀ ĐÁNH GIÁ MÔ HÌNH AI

6.1. Vai trò của model AI train trong hệ thống

Một điểm quan trọng của đề án là làm rõ lợi ích của việc dùng model AI train. Nếu hệ thống chỉ phân tích độ tương phản, độ sắc nét hoặc vài chỉ số ảnh, người dùng khó thấy sự khác biệt giữa xử lý ảnh thông thường và AI. Model AI train giúp hệ thống học từ dữ liệu có nhãn để đưa ra dự đoán như "nghi gãy xương", "bình thường", "nghi viêm/thoái hóa khớp" hoặc "nghi loãng xương".

Tuy nhiên, model phải có quy tắc tin cậy rõ ràng. Nếu quá dễ dãi, hệ thống có thể dán nhãn fracture cho mọi ảnh với confidence 100%. Nếu quá thận trọng, ảnh gãy rõ cũng chỉ hiện "analysis only", làm mất ý nghĩa của AI. Vì vậy, hệ thống cần cân bằng:

- Cho phép model đưa ra gợi ý khi bằng chứng đủ mạnh.
- Hiệu chỉnh confidence để tránh tự tin giả.
- Chặn kết luận khi ảnh không phù hợp hoặc nằm ngoài vùng dữ liệu training.
- Minh bạch về giới hạn của model demo.

6.2. Dữ liệu huấn luyện demo

Trong phạm vi đề án, hệ thống có script tạo dataset demo trong *examples/demo_bone_dataset*. Dataset được chia theo thư mục lớp:

```
demo_bone_dataset/  
  normal/  
  fracture/  
  arthritis/  
  osteoporosis/
```

Mỗi ảnh được sinh nhằm mô phỏng một số đặc trưng hình ảnh cơ bản. Dataset này giúp kiểm tra pipeline từ tạo dữ liệu, train model, lưu model, load model đến suy luận. Tuy nhiên, dataset demo không đại diện cho dữ liệu y khoa thật. Vì vậy, kết quả đánh giá chỉ có ý nghĩa kỹ thuật, không có ý nghĩa lâm sàng.

Đối với dữ liệu thực tế, hệ thống hỗ trợ hai định dạng:

- Folder per class: mỗi lớp là một thư mục chứa ảnh.
- CSV: mỗi dòng gồm đường dẫn ảnh và nhãn.

6.3. Huấn luyện lightweight feature model

Feature model được huấn luyện bằng script:

```
python scripts/train_bone_feature_model.py --data-dir examples/demo_bone_dataset --out-dir models/bone_feature_demo
```

Quy trình huấn luyện:

- Đọc ảnh theo từng lớp.
- Tiền xử lý ảnh giống backend inference.
- Trích xuất feature vector.
- Chia train/test.
- Huấn luyện mô hình nhẹ.
- Tính metrics trên tập test.
- Lưu model ra file JSON.

File model gồm labels, tham số, vector mẫu hoặc centroid, metrics và ghi chú. Backend tự động tìm file:

```
models/bone_feature_demo/bone_feature_model.json
```

Nếu file tồn tại, backend load model và trả *model_prediction* khi đủ điều kiện. Nếu không tồn tại, backend trả *analysis_only*.

6.4. Huấn luyện CNN/TorchScript model

Hệ thống cũng chuẩn bị hướng huấn luyện CNN bằng PyTorch:

```
python scripts/train_bone_model.py --data-dir dataset --out-dir models/bone_cnn --epochs 20
```

Kết quả huấn luyện gồm:

- *bone_model.pt*: TorchScript model để inference.
- *bone_model.pth*: checkpoint PyTorch.
- *bone_model.labels.json*: danh sách nhãn.
- *metrics.json*: kết quả đánh giá.

Khi có model *.pt*, có thể cấu hình:

MEDICAL_IMAGE_MODEL_PATH=models/bone_cnn/bone_model.pt

MEDICAL_IMAGE_MODEL_LABELS=normal,fracture,arthritis,osteoporosis,other

Sau đó restart backend để sử dụng model mới. Đây là hướng phát triển quan trọng nếu có dữ liệu thật.

6.5. Hiệu chỉnh confidence

Lúc mới đưa model KNN vào suy luận, có một rủi ro rất lớn: model thường xuyên trả về độ tự tin (confidence) 100% chỉ vì 3 điểm lân cận đều mang nhãn fracture. Trong y khoa, một con số 100% tuyệt đối là cực kỳ nguy hiểm và dễ gây hiểu nhầm. Nhận thấy điều này, mình quyết định không thể dùng trực tiếp xác suất thô (raw confidence). Thay vào đó, mình thiết kế lại công thức tính độ tin cậy bằng cách ép model phải xét thêm khoảng cách giữa top 1 và top 2 (margin) cũng như mức độ tương đồng với dữ liệu training (proximity). Nói cách khác, để hệ thống dám hiển thị con số 95% cho một ca gãy xương, tám ảnh đầu vào không chỉ cần raw confidence cao mà còn phải thực sự 'khớp' với phân phối chuẩn. Nếu đưa vào một tám ảnh mờ, nhiễu, hệ thống bị ép tự động hạ độ tin cậy xuống trạng thái 'analysis only'.

6.6. Quy tắc model_prediction và analysis_only

Quy tắc quyết định:

Nếu model chưa sẵn sàng:

status = analysis_only

Nếu ảnh không đạt chất lượng tối thiểu:

status = analysis_only

Nếu ảnh nằm xa phân phối training:

status = analysis_only

Nếu nhãn fracture có raw confidence rất cao, margin lớn và feature phù hợp:

status = model_prediction

top_label = fracture

confidence = calibrated_confidence

Nếu nhãn khác nhưng model demo chưa đủ tin cậy:

status = analysis_only

Nếu dùng model thật đã đánh giá tốt:

status = model_prediction theo threshold cấu hình

Quy tắc này giúp hệ thống thể hiện lợi ích của AI train mà không đánh đồng model demo với công cụ chẩn đoán.

6.7. Đánh giá model bằng metrics

Module đánh giá sử dụng các công thức:

Accuracy = Số dự đoán đúng / Tổng số mẫu

Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

F1 = $2 * Precision * Recall / (Precision + Recall)$

Macro-F1 = Trung bình F1 của tất cả các lớp

Weighted-F1 = Trung bình F1 có trọng số theo số mẫu từng lớp

Trong đồ án, macro-F1 được dùng để chọn model tốt nhất vì nó ít bị lệ thuộc vào mất cân bằng lớp. Nếu một model chỉ dự đoán tốt lớp normal nhưng bỏ sót fracture, macro-F1 sẽ phản ánh vấn đề này rõ hơn accuracy.

6.8. Đánh giá chatbot RAG

Đối với chatbot, đánh giá không chỉ là đúng/sai theo nhãn mà còn gồm:

- Hệ thống có hiểu đúng chủ đề câu hỏi không.
- Dữ liệu truy xuất có liên quan không.
- Câu trả lời có bám câu hỏi không.
- Câu trả lời có cảnh báo an toàn không.
- Hệ thống có tránh trả lời nhầm bệnh không.

Một số ca kiểm thử:

Câu hỏi	Kết quả mong đợi
---------	------------------

Triệu chứng của gout	Trả lời về gout, không trả cúm/sốt xuất huyết
Dấu hiệu mang thai	Trả lời về thai kỳ, không trả bệnh nhiễm trùng
Thoái hóa cột sống cổ	Trả lời về đau cổ, cứng cổ, tê lan tay
Tìm thông tin bệnh tiểu đường	Trả lời về tiểu đường, glucose/HbA1c

Sau khi bổ sung nhận diện chủ đề và stop words, chatbot giảm đáng kể lỗi truy xuất sai bệnh.

6.9. Đánh giá fusion

Fusion được đánh giá bằng cách so sánh:

- Image-only: dự đoán từ ảnh.
- Clinical-only: dự đoán từ lâm sàng.
- Lab-only hoặc clinical-lab.
- Multimodal: kết hợp nhiều nguồn.

Kỳ vọng là multimodal giúp giảm sai số trong các ca mà một nguồn dữ liệu không đủ rõ. Ví dụ, ảnh X-quang gãy kín đáo nhưng người bệnh đau khu trú, sưng nề sau chấn thương và hạn chế vận động thì fusion score tăng, giúp hệ thống ưu tiên ca này cho bác sĩ xem lại.

6.10. Giới hạn của model trong đề án

Phải thừa nhận thẳng thắn: model trong đề án này chưa thể mang ra viện xài thật. Tập dataset demo mình dựng lên chủ yếu để chứng minh luồng kiến trúc (pipeline) chạy thông suốt từ đầu tới cuối. Do thiếu nguồn dữ liệu X-quang chuẩn được bác sĩ chuyên khoa gán nhãn và số lượng mẫu còn quá hẻo, việc dùng feature model nhẹ nhàng thay vì mạng CNN phức tạp là một sự đánh đổi có chủ ý. Cái 'ăn tiền' của hệ thống nằm ở chỗ kiến trúc đã dọn sẵn đường: khi có trong tay tập data đủ lớn và xịn, chỉ cần cấu hình lại file .pt là hệ thống tự động scale lên được ngay.

CHƯƠNG 7. THỬ NGHIỆM HỆ THỐNG VÀ KẾT QUẢ ĐẠT ĐƯỢC

7.1. Môi trường thử nghiệm

Hệ thống được thử nghiệm trên môi trường phát triển local:

Thành phần	Công nghệ	Cổng
Chat frontend	Next.js	3001
MCP server	Next.js	3000
Medical backend	FastAPI	8000
PostgreSQL	Docker/PostgreSQL	5432
pgAdmin	Docker	5050

Các lệnh chạy trong Git Bash:

```
cd /c/Users/User/Desktop/doantotnghiep/Chat
npm run dev
cd /c/Users/User/Desktop/doantotnghiep/mcp-server
npm run dev
cd /c/Users/User/Desktop/doantotnghiep/mcp-server
./venv/Scripts/python.exe scripts/run_backend.py --reload
```

7.2. Kiểm thử chatbot

Các câu hỏi thử nghiệm:

- "Triệu chứng của gout"
- "Tìm thông tin bệnh gout"
- "Dấu hiệu mang thai"
- "Bệnh thoái hóa cột sống cổ có biểu hiện gì?"
- "Tìm thông tin bệnh tiểu đường"

Kết quả mong đợi:

- Hệ thống trả lời đúng chủ đề.
- Không lấy nhầm dữ liệu bệnh khác.

- Câu trả lời có cấu trúc dễ đọc.
- Có khuyến cáo tham khảo bác sĩ khi cần.

Khâu test pipeline RAG thực sự phơi bày rất nhiều lỗ hổng logic của AI. Đỉnh điểm là lúc mình test thử câu 'dấu hiệu mang thai', thuật toán search lại bốc ngay bài... 'sốt xuất huyết' đập thẳng vào mặt user, chỉ vì cả hai document đều chứa từ khóa 'dấu hiệu'. Trải nghiệm xong ca này, mình nhận ra dùng search từ khóa thuần túy trong y tế là tự sát. Ngay lập tức, mình phải đập đi code lại luồng truy xuất, chèn thêm một layer 'filter chủ đề' cứng ngắc ngay từ đầu. Bắt buộc thuật toán phải phân loại (classify) được cụm bệnh lý trước, khóa chặt phạm vi tìm kiếm lại rồi mới được phép query dữ liệu, từ đó chặn đứng hoàn toàn căn bệnh 'thấy chữ nào giống là bốc' của bot.

7.3. Kiểm thử phân tích phim xương

Lúc test module phân tích ảnh, thay vì chỉ đưa ảnh đẹp vào để lấy kết quả 'nghi gãy xương' mượt mà, mình cố tình tìm cách 'phá' hệ thống bằng các ca biên (edge cases). Mình ném đủ loại rác vào: ảnh kéo tối om, ảnh cháy sáng lóa, thậm chí ném cả ảnh không liên quan gì đến phim xương. Rất may là các chốt chặn an toàn hoạt động đúng như thiết kế. Thay vì cố đoán bừa và ném ra kết quả sai bét, hệ thống tự động fallback về trạng thái 'analysis only', kiên quyết từ chối đưa ra kết luận y khoa. Đây chính là cái phanh an toàn cực kỳ cần thiết khi phát triển ứng dụng AI cho lĩnh vực nhạy cảm này.

7.4. Kiểm thử xét nghiệm

Hệ thống được thử bằng các chỉ số máu/nước tiểu thường gặp. Ví dụ:

Chỉ số	Giá trị	Kỳ vọng
WBC	cao	Gợi ý bạch cầu tăng, có thể liên quan viêm/nhiễm
HGB	thấp	Gợi ý thiếu máu hoặc cần đổi chiều
Glucose	cao	Gợi ý tăng đường huyết
Protein niệu	dương tính	Gợi ý bất thường nước tiểu

Kết quả trả về gồm danh sách item, trạng thái, mức độ, diễn giải và khuyến nghị. Các chỉ số chưa nhận diện được được đưa vào danh sách unrecognized lines để người dùng biết cần kiểm tra lại.

7.5. Kiểm thử fusion

Một ca thử nghiệm fusion:

- Ảnh: X-quang cổ tay nghi gãy xương.
- Triệu chứng: đau nhiều, sưng, hạn chế vận động sau chấn thương.
- Xét nghiệm: không có hoặc không nổi bật.

Kết quả mong đợi:

- Image score cao do model dự đoán fracture.
- Clinical score tăng do triệu chứng phù hợp.
- Fusion score ở mức medium hoặc high tùy dữ liệu.
- Báo cáo gợi ý cần bác sĩ đọc phim gốc và đối chiếu vị trí đau.

Fusion giúp kết quả không phụ thuộc hoàn toàn vào ảnh. Nếu ảnh mờ nhưng triệu chứng mạnh, hệ thống vẫn có thể cảnh báo cần xem lại. Nếu ảnh có dấu hiệu nhẹ nhưng lâm sàng không phù hợp, risk level có thể không quá cao.

7.6. Kiểm thử đánh giá model

File CSV đánh giá có thể có dạng:

```
y_true,y_pred,model_name  
fracture,fracture,image_ai  
normal,normal,image_ai  
fracture,normal,image_ai  
fracture,fracture,multimodal  
normal,normal,multimodal  
fracture,fracture,multimodal
```

Hệ thống đọc CSV, nhóm theo *model_name*, tính metrics và trả model có macro-F1 cao nhất. Kết quả này có thể dùng trong báo cáo để so sánh hiệu quả trước/sau khi dùng AI hoặc so sánh image-only với multimodal.

7.7. Kiểm thử kỹ thuật

Các kiểm thử kỹ thuật đã thực hiện:

- Chạy TypeScript compiler cho Chat để kiểm tra lỗi kiểu.
- Chạy Python compile cho backend model.
- Kiểm tra API chat không treo stream.
- Kiểm tra API phân tích ảnh trả kết quả đúng định dạng.
- Kiểm tra backend phân biệt model_prediction và analysis_only.
- Kiểm tra các ảnh không phù hợp không bị gán fracture 100%.

7.8. Kết quả đạt được

Hệ thống đã đạt các kết quả:

- Xây dựng được ứng dụng web có nhiều module y khoa.
- Tích hợp được chatbot AI với provider cấu hình qua API key.
- Cài đặt được pipeline RAG/MCP ở mức demo, có xử lý lỗi truy xuất sai chủ đề.
- Cài đặt được backend phân tích ảnh y tế và model ảnh.
- Cài đặt được module xét nghiệm và fusion.
- Cài đặt được module đánh giá model.
- Có cơ chế lưu lịch sử upload, mở lại file và so sánh kết quả.
- Có cảnh báo an toàn và phân biệt kết quả hỗ trợ với chẩn đoán.

7.9. Hạn chế

Các hạn chế hiện tại:

- Dữ liệu huấn luyện ảnh chủ yếu là demo, chưa đủ để đánh giá lâm sàng.
- Chưa có tập validation độc lập từ bệnh viện.
- RAG nội bộ còn ở mức file Excel và từ khóa, chưa dùng vector database đầy đủ.
- Chưa có cơ chế trích dẫn nguồn y khoa chuẩn cho mọi câu trả lời.
- Giao diện còn cần hoàn thiện thêm về tiếng Việt, encoding và trải nghiệm.
- Chưa có phân quyền chi tiết cho từng vai trò.

- Chưa triển khai production với HTTPS, logging và bảo mật dữ liệu y tế.

CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1. Kết luận

Nhìn lại toàn bộ chặng đường, hệ thống trợ lý y khoa này tuy chạy ổn định và ghép nối thành công khá nhiều công nghệ "nặng đô" (từ RAG, MCP đến xử lý ảnh y tế trên backend FastAPI), nhưng mình hiểu rõ đây mới chỉ là vạch xuất phát. Cú "ăn tiền" lớn nhất của đề án không nằm ở việc show-off các mô hình AI xịn cỡ nào, mà là việc đập đi xây lại được những bài toán rất "đời" của dân Dev. Quá trình này buộc mình phải đổi mới và giải quyết triệt để tình trạng ảo giác từ khóa của chatbot bằng các bộ lọc chủ đề cứng. Đồng thời, hệ thống cũng thiết lập được một bộ quy tắc an toàn nghiêm ngặt để kìm hãm sự tự tin thái quá của model phân tích ảnh, xóa bỏ hoàn toàn hai thái cực cực đoan: hoặc nhắm mắt dán nhãn bừa bãi, hoặc chặn đứng mọi kết quả do quá nhát cáy. Tuy nhiên, cần phải khẳng định lại một cách minh bạch: với giới hạn về tập dữ liệu huấn luyện và thời gian của một đề án sinh viên, toàn bộ hệ thống hiện tại vẫn đang đóng vai trò là một prototype phục vụ mục đích nghiên cứu và học tập. Nó tuyệt đối không được sử dụng để thay thế các chẩn đoán lâm sàng của bác sĩ chuyên khoa.

8.2. Đóng góp của đề án

Thay vì sa đà vào việc liệt kê các tính năng bề nổi, giá trị thực tiễn lớn nhất mà đề án mang lại nằm ở tư duy thiết kế kiến trúc hệ thống. Đề án đã đề xuất và chứng minh tính khả thi của một luồng xử lý dữ liệu phức hợp, nơi Chatbot, cơ chế RAG, chuẩn giao tiếp MCP và Backend y khoa không hoạt động rời rạc mà được tích hợp chặt chẽ vào một thể thống nhất. Mình đã hiện thực hóa được một pipeline đi từ đầu đến cuối (end-to-end): từ lúc người dùng upload một tấm phim xương thô, qua các bước tiền xử lý, đánh giá chất lượng, trích xuất đặc trưng, cho đến khi xuất ra kết quả dự đoán kèm theo mức độ tin cậy đã được hiệu chỉnh (confidence calibration). Không dừng lại ở hình ảnh, việc xây dựng thành công module đọc xét nghiệm và áp dụng thuật toán fusion đa phương thức đã cung cấp một góc nhìn toàn diện hơn hẳn, mô phỏng lại cách các bác sĩ kết hợp nhiều tín hiệu lâm sàng để đưa ra quyết định. Quan trọng hơn cả, việc thiết lập một module đánh giá định lượng bằng các chuẩn metrics (như Macro-F1) đã giúp quá trình tinh chỉnh model có cơ sở khoa học rõ ràng, thoát khỏi việc tinh chỉnh dựa trên cảm tính.

8.3. Hướng phát triển

Làm project thực tế thì không thể tránh khỏi "nợ kỹ thuật" (technical debt). Để hệ thống có thể scale up (mở rộng) và tiệm cận với môi trường production của một bệnh viện, bài toán đầu tiên cần giải quyết là phải vứt bỏ bộ feature model nhẹ hều hiện tại. Tham vọng lớn nhất là xin được cấp phép truy cập vào các tập dataset X-quang chuẩn y khoa để train trực tiếp một mạng CNN hoặc Vision Transformer đẳng hoàng. Thứ hai, cơ chế RAG dựa trên luồng search từ khóa hiện tại vẫn còn khá "com". Để giải quyết triệt để rủi ro sai lệch ngữ nghĩa, hệ thống cần được bổ sung thêm một Vector Database chuyên dụng (như Milvus hay Pinecone) kết hợp với các embedding model được fine-tune riêng cho y tế. Cuối cùng, để đưa ra thực tế, toàn bộ hệ thống phân quyền, logging, mã hóa đầu cuối và chuẩn hóa bảo mật dữ liệu theo chuẩn y tế (như HIPAA) sẽ là những tính năng bắt buộc phải được implement.

8.4. Kết luận cuối

Hành trình hoàn thiện đề án này đã thay đổi hoàn toàn góc nhìn của mình về việc ứng dụng Trí tuệ nhân tạo. AI chắc chắn mang lại giá trị to lớn trong việc tự động hóa sàng lọc, tra cứu và tổng hợp thông tin. Nhưng giá trị thực sự của một hệ thống y tế không nằm ở việc tạo ra một cỗ máy thay thế con người, mà là tạo ra một trợ lý đủ tin cậy để nhân viên y tế có thêm góc nhìn tham khảo, giúp bệnh nhân tiếp cận thông tin nhanh chóng hơn. Điều quan trọng nhất rút ra từ hàng trăm giờ code và debug là: một hệ thống AI sinh ra cho lĩnh vực y tế cần phải cân bằng được hai yếu tố – nó phải vừa đủ thông minh để mang lại sự hữu ích, nhưng cũng phải bị trói buộc bởi những quy tắc đủ thận trọng để đảm bảo an toàn tuyệt đối cho người dùng cuối.

TÀI LIỆU THAM KHẢO

- [1] Next.js Documentation, <https://nextjs.org/docs>
- [2] React Documentation, <https://react.dev>
- [3] Tailwind CSS Documentation, <https://tailwindcss.com/docs>
- [4] FastAPI Documentation, <https://fastapi.tiangolo.com>
- [5] Pydantic Documentation, <https://docs.pydantic.dev>
- [6] Vercel AI SDK Documentation, <https://sdk.vercel.ai/docs>
- [7] Model Context Protocol Documentation, <https://modelcontextprotocol.io>
- [8] PostgreSQL Documentation, <https://www.postgresql.org/docs>
- [9] Drizzle ORM Documentation, <https://orm.drizzle.team>
- [10] OpenCV Documentation, <https://docs.opencv.org>
- [11] PyTorch Documentation, <https://pytorch.org/docs>
- [12] World Health Organization, Ethics and governance of artificial intelligence for health.
- [13] Esteva, A. et al., A guide to deep learning in healthcare, Nature Medicine.
- [14] Rajpurkar, P. et al., CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning.
- [15] Lewis, P. et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

PHỤ LỤC A. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY HỆ THỐNG

A.1. Chạy PostgreSQL

```
cd /c/Users/User/Desktop/doantotnghiep/postgres
docker-compose up -d
```

A.2. Chạy Chat frontend

```
cd /c/Users/User/Desktop/doantotnghiep/Chat
npm install
npm run dev
```

Truy cập:

<http://localhost:3001>

A.3. Chạy MCP server

```
cd /c/Users/User/Desktop/doantotnghiep/mcp-server
npm install
npm run dev
```

Truy cập:

<http://localhost:3000>

A.4. Chạy FastAPI backend trong Git Bash

```
cd /c/Users/User/Desktop/doantotnghiep/mcp-server
./venv/Scripts/python.exe scripts/run_backend.py --reload
```

Truy cập Swagger:

<http://localhost:8000/docs>

A.5. Cấu hình biến môi trường Chat

File:

Chat/.env.local

Ví dụ:

```
GOOGLE_API_KEY=your_google_key
ANTHROPIC_API_KEY=
GROQ_API_KEY=
```

NEXT_PUBLIC_MEDICAL_IMAGE_API_URL=http://localhost:8000

MEDICAL_IMAGE_API_URL=http://localhost:8000

Không đưa API key thật lên GitHub hoặc ảnh chụp công khai.

PHỤ LỤC B. MẪU CSV ĐÁNH GIÁ

```
y_true,y_pred,model_name  
fracture,fracture,image_ai  
normal,normal,image_ai  
arthritis,fracture,image_ai  
osteoporosis,osteoporosis,image_ai  
fracture,fracture,multimodal  
normal,normal,multimodal  
arthritis,arthritis,multimodal  
osteoporosis,osteoporosis,multimodal
```

PHỤ LỤC C. GỢI Ý DANH MỤC HÌNH ẢNH CẦN CHÈN

Mã hình	Nội dung
Hình 4.1	Sơ đồ kiến trúc tổng thể
Hình 5.1	Giao diện chatbot
Hình 5.2	Giao diện upload phim xương
Hình 5.3	Kết quả nghi gãy xương
Hình 5.4	Giao diện xét nghiệm
Hình 5.5	Kết quả phân tích xét nghiệm
Hình 5.6	Giao diện fusion
Hình 5.7	Báo cáo fusion
Hình 5.8	Giao diện đánh giá model
Hình 5.9	Bảng metrics

PHỤ LỤC D. GỢI Ý BẢNG BIỂU

Mã bảng	Nội dung
Bảng 2.1	So sánh RAG và chatbot thường
Bảng 2.2	Các chỉ số đánh giá model
Bảng 3.1	Yêu cầu chức năng
Bảng 4.1	Các service và cổng chạy
Bảng 4.2	Schema bảng chats
Bảng 4.3	Schema bảng messages
Bảng 6.1	Các lớp model ảnh
Bảng 6.2	Quy tắc confidence
Bảng 7.1	Cá kiểm thử chatbot
Bảng 7.2	Cá kiểm thử ảnh
Bảng 7.3	Kết quả metrics

PHỤ LỤC E. GHI CHÚ AN TOÀN Y KHOA

Hệ thống trong đồ án chỉ phục vụ mục đích học tập, nghiên cứu và minh họa kỹ thuật. Các kết quả như "nghi gãy xương", "nguy cơ trung bình", "chỉ số bất thường" hoặc câu trả lời chatbot không phải là chẩn đoán y khoa cuối cùng. Người dùng cần tham khảo bác sĩ hoặc cơ sở y tế, đặc biệt khi có triệu chứng nặng, đau nhiều, khó thở, mất ý thức, sốt cao kéo dài, chảy máu, yếu liệt, biến dạng chi, đau ngực hoặc các dấu hiệu nguy hiểm khác.

PHỤ LỤC F. ĐẶC TẢ USE CASE CHI TIẾT

F.1. Use case UC01 - Đăng ký tài khoản y khoa

Mục tiêu của use case này là cho phép người dùng tạo tài khoản để sử dụng các chức năng có lưu lịch sử như upload phim xương, xét nghiệm, fusion và đánh giá model. Người dùng nhập tên đăng nhập, mật khẩu và họ tên. Hệ thống kiểm tra độ dài tên đăng nhập, độ dài mật khẩu, sau đó hash mật khẩu và lưu thông tin vào cơ sở dữ liệu backend. Nếu tên đăng nhập đã tồn tại, hệ thống trả lỗi và yêu cầu người dùng chọn tên khác. Nếu đăng ký thành công, hệ thống trả access token để người dùng có thể tiếp tục thao tác mà không cần đăng nhập lại ngay.

Tiền điều kiện của use case là backend FastAPI đang chạy và cơ sở dữ liệu SQLite có thể ghi dữ liệu. Hậu điều kiện là một bản ghi người dùng mới được tạo, mật khẩu không lưu dạng plain text và frontend lưu token ở local storage. Luồng ngoại lệ gồm: tên đăng nhập quá ngắn, mật khẩu quá ngắn, trùng username, backend không phản hồi hoặc token không hợp lệ.

F.2. Use case UC02 - Đăng nhập hệ thống y khoa

Người dùng nhập username và password. Frontend gửi thông tin đến endpoint đăng nhập. Backend tìm người dùng theo username, kiểm tra hash mật khẩu, sau đó trả về access token nếu hợp lệ. Token được dùng cho các request tiếp theo qua header Authorization dạng Bearer token. Nếu đăng nhập thất bại, hệ thống không tiết lộ username hay password sai cụ thể để giảm rủi ro bảo mật.

Use case này giúp hệ thống gắn kết quả upload với đúng người dùng. Ví dụ, khi người dùng upload phim xương, hệ thống lưu file vào thư mục theo user id và lưu metadata trong bảng upload history. Khi sang module fusion, người dùng có thể tự động nạp lại ảnh hoặc xét nghiệm gần nhất.

F.3. Use case UC03 - Hỏi thông tin bệnh lý bằng chatbot

Người dùng nhập câu hỏi tự nhiên như "triệu chứng của gout", "dấu hiệu mang thai" hoặc "thoái hóa cột sống cổ có biểu hiện gì". Hệ thống gửi câu hỏi đến API chat. API

phân tích câu hỏi, kiểm tra đây có phải câu hỏi y khoa hay không, nhận diện chủ đề nếu có, sau đó truy xuất dữ liệu nội bộ hoặc gọi mô hình ngôn ngữ lớn để trả lời.

Điểm cần chú ý là câu hỏi y khoa thường chứa nhiều từ chung. Nếu chỉ tìm kiếm theo số lần xuất hiện từ khóa, hệ thống có thể trả nhầm bệnh. Vì vậy, use case này yêu cầu hệ thống lọc chủ đề trước khi chọn dữ liệu nội bộ. Nếu câu hỏi về gout, dữ liệu cúm mùa hoặc sốt xuất huyết không được chọn dù có cùng từ "triệu chứng". Nếu không tìm được dữ liệu nội bộ phù hợp, hệ thống dùng fallback y khoa an toàn thay vì cố ép dữ liệu sai.

F.4. Use case UC04 - Upload và phân tích phim xương

Người dùng chọn file phim xương, chọn modality và nhập vùng xương. Frontend kiểm tra định dạng và dung lượng file, sau đó gửi form-data đến backend qua API route. Backend đọc file, tiền xử lý ảnh, tính quality metrics, trích xuất đặc trưng và gọi model ảnh. Kết quả trả về gồm metadata, preprocessing, quality, features, prediction và safety note.

Nếu model đủ điều kiện, hệ thống hiển thị "Model gợi ý" cùng nhãn và độ tin cậy. Nếu model chưa đủ điều kiện, hệ thống hiển thị "Analysis only" và chỉ mô tả đặc trưng ảnh. Use case này có ý nghĩa quan trọng trong đề án vì thể hiện lợi ích của model AI train. Với ảnh gãy xương rõ, model cần đưa ra gợi ý nghi gãy xương, nhưng vẫn không được tự tin tuyệt đối hoặc thay thế chẩn đoán bác sĩ.

F.5. Use case UC05 - Xem lịch sử upload phim xương

Sau khi đăng nhập, người dùng có thể xem các ảnh/phim đã upload trước đó. Hệ thống gọi endpoint upload history, lọc theo loại image và hiển thị danh sách gồm tên file, thời gian, modality, vùng xương và kết quả phân tích. Người dùng có thể mở lại file đã upload hoặc chọn hai lần upload để so sánh.

Chức năng này giúp người dùng theo dõi sự thay đổi giữa các lần chụp hoặc kiểm thử nhiều ảnh khác nhau trong quá trình demo. Về mặt kỹ thuật, nó chứng minh hệ thống không chỉ xử lý request tức thời mà còn có khả năng lưu vết dữ liệu và kết quả AI.

F.6. Use case UC06 - Nhập và đọc kết quả xét nghiệm

Người dùng nhập tuổi, giới tính và các chỉ số xét nghiệm. Mỗi dòng gồm tên chỉ số, giá trị, đơn vị và nhóm xét nghiệm. Hệ thống chuẩn hóa tên chỉ số, tìm khoảng tham chiếu, so sánh giá trị và trả trạng thái. Ví dụ, glucose cao có thể được gán trạng thái high, hemoglobin thấp có thể được gán trạng thái low, protein niệu dương tính có thể được gán trạng thái abnormal.

Kết quả xét nghiệm được trình bày dưới dạng danh sách item, summary, abnormal count, urgent count và recommended next steps. Hệ thống không kết luận bệnh cuối cùng mà chỉ giải thích ý nghĩa tham khảo của từng chỉ số. Nếu một chỉ số không nhận diện được, hệ thống đưa vào danh sách unrecognized lines để người dùng biết cần kiểm tra lại.

F.7. Use case UC07 - Upload file xét nghiệm

Người dùng có thể upload file xét nghiệm thay vì nhập thủ công. Backend trích xuất văn bản từ file, sau đó đưa văn bản vào module phân tích xét nghiệm. Use case này phù hợp với tình huống người dùng có phiếu xét nghiệm dạng text, PDF hoặc ảnh đã được OCR trước.

Trong phạm vi đề án, khả năng đọc file phục vụ minh họa luồng xử lý. Khi triển khai thực tế, cần bổ sung OCR mạnh hơn, chuẩn hóa layout phiếu xét nghiệm theo từng bệnh viện và kiểm tra thủ công các dòng không nhận diện được.

F.8. Use case UC08 - Chạy fusion đa phương thức

Người dùng nhập hoặc nạp lại ảnh, triệu chứng, tiền sử, điểm đau và xét nghiệm. Backend phân tích từng nguồn dữ liệu, sau đó tính fusion score. Kết quả gồm predicted label, risk level, confidence, danh sách signals và structured report.

Use case này thể hiện điểm mạnh của hệ thống so với các module rời rạc. Trong thực tế, bác sĩ không đọc ảnh tách khỏi triệu chứng. Một phim X-quang cần được đối chiếu với vị trí đau, cơ chế chấn thương, thời điểm chụp và khả năng vận động. Fusion giúp hệ thống mô phỏng một phần tư duy tổng hợp này ở mức hỗ trợ.

F.9. Use case UC09 - Đánh giá model bằng CSV

Người dùng chuẩn bị file CSV có nhãn thật, nhãn dự đoán và tên model. Hệ thống đọc file, nhóm theo model name và tính metrics. Kết quả trả về giúp so sánh nhiều phương pháp như before AI, image AI, clinical AI và multimodal.

Use case này rất quan trọng trong báo cáo tốt nghiệp vì nó cung cấp bằng chứng định lượng. Thay vì chỉ nói model "có vẻ tốt", người thực hiện có thể đưa bảng accuracy, macro-F1 và confusion matrix vào phần thực nghiệm. Nếu multimodal có macro-F1 cao hơn image-only, hệ thống có cơ sở để chứng minh lợi ích của fusion.

F.10. Use case UC10 - Lưu prediction run thủ công

Trong quá trình demo, có những ca người dùng muốn lưu nhanh kết quả mà không chuẩn bị CSV. Module đánh giá cho phép nhập case code, nhãn thật, dự đoán trước AI, image AI, clinical AI, multimodal và ghi chú. Hệ thống lưu prediction run vào database. Dữ liệu này có thể được xem lại, bổ sung vào CSV hoặc dùng để minh họa quá trình kiểm thử.

PHỤ LỤC G. KỊCH BẢN KIỂM THỬ CHI TIẾT

G.1. Nhóm kiểm thử chatbot

Kịch bản 1: Người dùng hỏi "triệu chứng của gout". Kết quả mong đợi là hệ thống trả lời về đau khớp dữ dội, sưng nóng đỏ, thường gặp ở ngón chân cái, có thể tái phát và liên quan acid uric. Hệ thống không được trả lời cúm mùa, sốt xuất huyết hoặc bệnh không liên quan.

Kịch bản 2: Người dùng hỏi "dấu hiệu mang thai". Kết quả mong đợi là hệ thống trả lời về trễ kinh, buồn nôn, căng tức ngực, mệt mỏi, đi tiểu nhiều, thay đổi vị giác và khuyến nghị dùng que thử thai hoặc khám sản phụ khoa. Hệ thống không được trả lời sốt xuất huyết hoặc bệnh nhiễm trùng.

Kịch bản 3: Người dùng hỏi "bệnh thoái hóa cột sống cổ là gì". Kết quả mong đợi là hệ thống trả lời về thoái hóa đĩa đệm/đốt sống cổ, đau cổ, cứng cổ, đau lan vai gáy, tê tay, yếu tay nếu chèn ép rễ thần kinh và khuyến nghị khám chuyên khoa nếu có dấu hiệu nặng.

Kịch bản 4: Người dùng hỏi một câu không thuộc y khoa. Hệ thống có thể trả lời theo model AI thông thường hoặc hướng dẫn người dùng hỏi về chức năng hệ thống. Hệ thống không nên tự gán chủ đề bệnh lý khi không có căn cứ.

Kịch bản 5: Người dùng hỏi câu có lỗi chính tả hoặc không dấu, ví dụ "trieu chung gout". Hệ thống cần chuẩn hóa tiếng Việt và vẫn nhận diện được chủ đề gout.

G.2. Nhóm kiểm thử upload ảnh

Kịch bản 6: Người dùng upload ảnh X-quang cổ tay có gãy xương rõ. Kết quả mong đợi là *model_prediction*, top label là *fracture*, giao diện hiển thị "Model gợi ý: nghi gãy xương", confidence cao và có cảnh báo cần bác sĩ đối chiếu phim gốc.

Kịch bản 7: Người dùng upload ảnh quá tối. Kết quả mong đợi là hệ thống trả *analysis_only* hoặc cảnh báo chất lượng ảnh kém. Hệ thống không được kết luận gãy xương chỉ vì ảnh có vùng đen/trắng bất thường.

Kịch bản 8: Người dùng upload ảnh quá sáng hoặc bị cháy sáng. Kết quả mong đợi là *quality warning* và không đưa ra kết luận bệnh lý nếu thông tin ảnh không đủ.

Kịch bản 9: Người dùng upload ảnh không phải phim xương. Kết quả mong đợi là hệ thống không tự tin dự đoán fracture. Nếu model không đủ điều kiện, kết quả phải là analysis only.

Kịch bản 10: Người dùng upload file sai định dạng. Frontend phải báo lỗi trước khi gửi hoặc backend trả lỗi rõ ràng.

Kịch bản 11: Người dùng upload file quá dung lượng. Hệ thống cần từ chối và thông báo giới hạn file.

G.3. Nhóm kiểm thử xét nghiệm

Kịch bản 12: Người dùng nhập glucose cao. Hệ thống cần đánh dấu high, diễn giải tăng đường huyết và khuyến nghị đối chiếu với bối cảnh nhịn đói, HbA1c hoặc bác sĩ.

Kịch bản 13: Người dùng nhập hemoglobin thấp. Hệ thống cần đánh dấu low, diễn giải có thể liên quan thiếu máu và khuyến nghị kiểm tra thêm nguyên nhân.

Kịch bản 14: Người dùng nhập protein niệu dương tính. Hệ thống cần đánh dấu abnormal hoặc positive, giải thích liên quan bất thường nước tiểu và khuyến nghị xét nghiệm lại hoặc khám nếu kéo dài.

Kịch bản 15: Người dùng nhập một chỉ số không có trong danh sách chuẩn. Hệ thống không được bỏ qua âm thầm, mà cần đưa vào unrecognized lines hoặc trạng thái unknown.

G.4. Nhóm kiểm thử fusion

Kịch bản 16: Ảnh gãy rõ, triệu chứng đau nhiều sau chấn thương. Fusion score cần tăng, risk level nên là medium hoặc high, báo cáo khuyến nghị bác sĩ đọc phim gốc.

Kịch bản 17: Ảnh không rõ nhưng triệu chứng mạnh. Hệ thống không kết luận chắc chắn, nhưng cần đưa ra khuyến nghị xem lại hoặc chụp bổ sung nếu nghi ngờ.

Kịch bản 18: Ảnh bình thường, triệu chứng nhẹ, xét nghiệm không bất thường. Fusion score nên thấp và báo cáo ghi chưa thấy tín hiệu nguy cơ nổi bật.

Kịch bản 19: Xét nghiệm có chỉ số viêm cao kèm triệu chứng sưng nóng. Fusion cần ghi nhận tín hiệu xét nghiệm và khuyến nghị đối chiếu viêm/nhiễm.

G.5. Nhóm kiểm thử đánh giá

Kịch bản 20: Upload CSV hợp lệ. Hệ thống cần đọc đúng records, nhóm theo model, tính metrics và trả model tốt nhất theo macro-F1.

Kịch bản 21: CSV thiếu cột bắt buộc. Hệ thống cần báo lỗi rõ ràng để người dùng sửa file.

Kịch bản 22: CSV có nhãn chưa từng xuất hiện. Hệ thống vẫn cần tạo confusion matrix đầy đủ từ tập hợp nhãn thật và nhãn dự đoán.

Kịch bản 23: Người dùng lưu prediction run thủ công. Hệ thống cần lưu record và hiển thị lại trong danh sách saved runs.

G.6. Nhóm kiểm thử bảo mật và phiên đăng nhập

Kịch bản 24: Người dùng chưa đăng nhập nhưng gọi API yêu cầu token. Hệ thống cần trả lỗi xác thực và frontend chuyển đến trang đăng nhập.

Kịch bản 25: Token sai hoặc hết hạn. Backend cần từ chối request và không trả dữ liệu của người dùng khác.

Kịch bản 26: Người dùng A upload file, người dùng B không được truy cập file đó nếu không có quyền.

G.7. Nhóm kiểm thử hiệu năng cơ bản

Kịch bản 27: Upload ảnh kích thước lớn nhưng trong giới hạn. Hệ thống cần resize và trả kết quả trong thời gian chấp nhận được.

Kịch bản 28: Chatbot trả lời streaming. Giao diện không bị treo sau khi thông báo đang tìm kiếm.

Kịch bản 29: Backend FastAPI restart. Frontend cần hiển thị lỗi rõ nếu backend chưa sẵn sàng, không để người dùng nghĩ hệ thống đã phân tích xong.

Kịch bản 30: MCP server chưa chạy. Chatbot cần có fallback hoặc thông báo lỗi cấu hình thay vì treo màn hình.

PHỤ LỤC H. ĐẶC TẢ API TÓM TẮT

H.1. API kiểm tra trạng thái

Endpoint *GET /health* dùng để kiểm tra backend y khoa. Response gồm trạng thái service, model đã sẵn sàng hay chưa, đường dẫn model và lỗi load model nếu có. API này hữu ích khi debug vì nhiều lỗi frontend thực chất đến từ backend chưa chạy hoặc model chưa load được.

Ví dụ response:

```
{
  "status": "ok",
  "model_ready": true,
  "model_path": "models/bone_feature_demo/bone_feature_model.json",
  "model_load_error": null
}
```

H.2. API đăng ký

Endpoint *POST /api/auth/register* nhận username, password và full name. Response trả access token và thông tin user public. Password được hash ở backend. API cần kiểm tra username trùng và độ dài mật khẩu.

H.3. API đăng nhập

Endpoint *POST /api/auth/login* nhận username và password. Nếu hợp lệ, backend trả access token. Frontend lưu token và gửi trong các request tiếp theo.

H.4. API phân tích ảnh

Endpoint *POST /api/images/analyze* nhận multipart form-data gồm file, modality và body_part. Response có cấu trúc:

- *metadata*: thông tin file và ảnh.
- *preprocessing*: thông tin chuẩn hóa ảnh.
- *quality*: chỉ số chất lượng.
- *features*: đặc trưng ảnh.
- *prediction*: kết quả model.

- *safety_note*: ghi chú an toàn.

Trường *prediction.status* có hai giá trị: *model_prediction* và *analysis_only*. Đây là trường quan trọng để frontend quyết định có hiển thị kết luận bệnh lý cụ thể hay không.

H.5. API phân tích xét nghiệm

Endpoint *POST /api/labs/analyze* nhận age, gender, danh sách values hoặc raw_text. Response gồm items, summary, abnormal_count, urgent_count, recommended_next_steps, safety_note, raw_text_preview và unrecognized_lines.

Mỗi item có tên chỉ số, giá trị, đơn vị, khoảng tham chiếu, trạng thái, mức độ và diễn giải. Thiết kế này giúp frontend có thể hiển thị bảng xét nghiệm chi tiết và phần tóm tắt tổng quan.

H.6. API phân tích file xét nghiệm

Endpoint *POST /api/labs/analyze-file* nhận file xét nghiệm, tuổi và giới tính. Backend trích xuất text rồi dùng lại pipeline phân tích xét nghiệm. API này làm giảm thao tác nhập tay, nhưng vẫn cần hiển thị các dòng chưa nhận diện để người dùng kiểm tra.

H.7. API phân tích lâm sàng

Endpoint *POST /api/clinical/analyze* nhận tuổi, giới, triệu chứng, tiền sử và clinical indicators. Response gồm normalized input, risk_items, summary, recommended_next_steps và safety_note. Module này là nguồn dữ liệu cho fusion.

H.8. API fusion

Endpoint *POST /api/multimodal/analyze* nhận file ảnh, modality, body_part, clinical_json và lab_json tùy chọn. Backend chạy các pipeline thành phần và trả response gồm image, clinical, lab, fusion_method, fusion_score, risk_level, predicted_label, confidence, signals, explanation, structured_report, recommended_next_steps và safety_note.

Thiết kế response có cả signals và structured_report để hệ thống vừa có dữ liệu định lượng vừa có phần giải thích tự nhiên cho người dùng.

H.9. API đánh giá

Endpoint đánh giá nhận records hoặc CSV. Mỗi record gồm `y_true`, `y_pred` và `model_name`. Backend tính metrics theo từng model. Response gồm danh sách model metrics, `best_model_by_macro_f1` và `comparison_summary`.

API này giúp đồ án có phần thực nghiệm rõ ràng, vì các model có thể được so sánh bằng cùng một chuẩn.

H.10. API upload history

Endpoint upload history trả các file người dùng đã upload. Response gồm `id`, `owner_user_id`, `upload_type`, `filename`, `content_type`, `file_path`, `file_size`, `modality`, `body_part`, `source_text`, `analysis`, `label_status`, `usable_for_training` và `created_at`. Trường `analysis` lưu JSON response của lần phân tích, giúp frontend hiển thị lại mà không cần chạy model lại ngay.

PHỤ LỤC I. QUẢN LÝ RỦI RO VÀ ĐỊNH HƯỚNG KIỂM ĐỊNH

I.1. Rủi ro chatbot trả lời sai

Rủi ro đầu tiên là chatbot trả lời sai bệnh hoặc trả lời quá tự tin. Nguyên nhân có thể đến từ dữ liệu nội bộ không đầy đủ, truy xuất sai dòng, model ngôn ngữ suy diễn hoặc câu hỏi người dùng mơ hồ. Biện pháp giảm thiểu gồm nhận diện chủ đề, lọc kết quả truy xuất, dùng fallback an toàn, hiển thị cảnh báo và khuyến nghị đi khám khi có dấu hiệu nguy hiểm.

Trong đồ án, lỗi thực tế từng gặp là hỏi gout nhưng hệ thống trả cúm mùa, hỏi dấu hiệu mang thai nhưng trả sốt xuất huyết. Đây là ví dụ tốt để đưa vào phần đánh giá vì nó cho thấy RAG không chỉ là "tìm dòng giống nhất" mà cần kiểm soát ngữ nghĩa. Sau khi bổ sung topic filter, hệ thống tránh được lỗi này trong các ca kiểm thử chính.

I.2. Rủi ro model ảnh quá tự tin

Model ảnh demo có thể trả xác suất 100% nếu thuật toán KNN thấy các láng giềng cùng lớp. Tuy nhiên, xác suất này không tương đương độ chắc chắn y khoa. Nếu hiển thị trực tiếp 100%, người dùng có thể hiểu nhầm rằng hệ thống chắc chắn tuyệt đối. Biện pháp giảm thiểu là hiệu chỉnh confidence, kiểm tra margin, kiểm tra khoảng cách đặc trưng và phân biệt model_prediction với analysis_only.

Quy tắc hiện tại cho phép model đưa ra gợi ý gãy xương khi bằng chứng mạnh, nhưng không cho phép mọi ảnh đều bị dán nhãn fracture. Đây là sự cân bằng cần thiết giữa tính hữu ích và an toàn.

I.3. Rủi ro model ảnh quá thận trọng

Ngược lại, nếu hệ thống luôn trả analysis only thì người dùng không thấy lợi ích của AI train. Trường hợp ảnh gãy xương rõ nhưng hệ thống không dám đưa ra gợi ý là một lỗi trải nghiệm và làm giảm giá trị đồ án. Vì vậy, policy cần có đường cho model được phát biểu khi dữ liệu đủ mạnh. Kết quả nên dùng ngôn ngữ "ngghi gãy xương" hoặc "model gợi ý", không dùng "chẩn đoán xác định".

I.4. Rủi ro dữ liệu huấn luyện không đại diện

Dataset demo không thể đại diện cho ảnh y tế thật. Model có thể hoạt động tốt trên ảnh demo nhưng kém trên ảnh bệnh viện. Để kiểm định nghiêm túc, cần thu thập dữ liệu từ nhiều thiết bị, nhiều vị trí xương, nhiều chất lượng ảnh, nhiều nhóm tuổi và có nhãn từ chuyên gia. Tập test cần tách biệt hoàn toàn với tập train.

I.5. Rủi ro bảo mật dữ liệu y tế

Ảnh phim, xét nghiệm và thông tin lâm sàng là dữ liệu nhạy cảm. Trong phạm vi đồ án, dữ liệu chạy local và phục vụ demo. Nếu triển khai thật, cần mã hóa dữ liệu lưu trữ, mã hóa đường truyền, phân quyền, audit log, chính sách xóa dữ liệu, backup và tuân thủ quy định pháp lý.

I.6. Định hướng kiểm định lâm sàng

Để hệ thống tiến gần ứng dụng thực tế, cần quy trình kiểm định gồm:

- Xác định bài toán lâm sàng cụ thể, ví dụ phát hiện gãy xương cổ tay trên X-quang.
- Thu thập dữ liệu có nhãn bởi ít nhất hai bác sĩ độc lập.
- Chia dữ liệu thành train, validation và test theo bệnh nhân, không theo ảnh.
- Huấn luyện model trên tập train.
- Chọn threshold trên validation.
- Báo cáo sensitivity, specificity, precision, recall, F1, AUC và confusion matrix trên test.
- Phân tích lỗi theo nhóm tuổi, giới, vị trí xương, chất lượng ảnh và thiết bị.
- So sánh model với bác sĩ hoặc workflow hiện tại.
- Đánh giá usability với người dùng thật.
- Xây dựng quy trình giám sát sau triển khai.

I.7. Định hướng nâng cấp RAG

RAG hiện tại có thể nâng cấp bằng cách dùng embedding và vector database. Dữ liệu bệnh lý nên được tách thành các đoạn nhỏ, mỗi đoạn có metadata về bệnh, nhóm triệu chứng, nguồn, ngày cập nhật và mức độ tin cậy. Khi truy xuất, hệ thống dùng hybrid

search gồm keyword filter và semantic search. Câu trả lời nên có trích dẫn nguồn để người dùng kiểm tra.

Ngoài ra, cần có bộ test regression cho chatbot. Mỗi khi thay đổi thuật toán truy xuất, hệ thống chạy lại các câu hỏi mẫu để đảm bảo không tái diễn lỗi trả nhảm bệnh.

I.8. Định hướng nâng cấp fusion

Fusion rule-weighted hiện tại minh họa được ý tưởng nhưng còn thủ công. Khi có dataset đủ lớn, có thể huấn luyện model fusion dựa trên đặc trưng ảnh, vector lâm sàng và xét nghiệm. Model fusion có thể là logistic regression, random forest, gradient boosting hoặc neural network nhẹ. Tuy nhiên, rule-based vẫn có giá trị vì dễ giải thích và phù hợp giai đoạn đầu.

I.9. Định hướng hoàn thiện giao diện

Giao diện cần bổ sung hướng dẫn ngắn tại từng module, cải thiện tiếng Việt, thêm biểu đồ trực quan cho metrics, thêm trạng thái backend/MCP đang chạy hay chưa, thêm thông báo khi API key thiếu hoặc model chưa load. Với module ảnh, có thể thêm overlay vùng nghi ngờ hoặc heatmap nếu dùng model có khả năng giải thích.

I.10. Kết luận phụ lục

Các rủi ro và định hướng trên cho thấy đồ án không chỉ dừng ở việc xây dựng demo chạy được, mà còn đặt nền tảng cho một hệ thống AI y khoa có trách nhiệm. Một hệ thống như vậy cần liên tục được kiểm thử, đánh giá, hiệu chỉnh và giám sát. AI chỉ thật sự hữu ích trong y tế khi nó được đặt trong quy trình có kiểm soát, có người chịu trách nhiệm chuyên môn và có minh bạch về giới hạn.